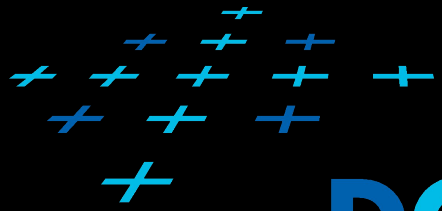
VIZ 09

VISUALIZATION OF RELATIONAL DATA

LADISLAV ČMOLÍK and PAVEL SLAVÍK

Department of Computer Graphics and Interaction
Faculty of Electrical Engineering at CTU in Prague



DCGI

Outline

- Relational data
- What is a relation?
- Visual encoding of relations
- Visual encoding of attributes
- Examples of relational data
- Tasks for relational data
- Visualization of hierarchies with containment
- Visualization of hierarchies and networks with graphs

Relational data

- Similar to tabular data, but contain **relations** (**links**) between items (**rows**) in one or more tables
  - To express hierarchy (Tree) or arbitrary relations (Network)
- Attributes
  - Both nodes and links can store values of several different attributes
  - Typically, the number of attributes is low
- Abstract data
  - Although the nodes can have spatial coordinates (they typically don't)
    - E.g., cities and airline flights between the cities
  - We cannot perform data enrichment
    - E.g., it does not make sense to interpolate between flights to different cities

What is a relation?

- Definition of a relation
  - A relation L over the sets X_1, …, X_k is a **subset** of their **cartesian product,** written $L \subseteq X_1 \times \ldots \times X_k$
- Unary relation
  - Is a subset of a set
  - E.g., We have set of students of visualization course S.
    Set of students that got A from the course is subset of S.
- Binary relation
  - Is a subset of a cartesian product of two sets
  - E.g., we have set of people P and set of addresses A.
    Cartesian product P x A contains all possible pairs (person, address).
    Binary relation is the subset containing only the pairs (person, address) where the person lives at the address.
- In this lecture we will focus mainly on binary relations

Example of 1:M binary relation

Key	Name	Lives at (FK)
P1	Alice	A2
P2	Ben	A2
P3	Chris	A1

Table of people P

Key	Address
A1	Long street 5
A2	Broad street 3
A3	Dead end 1

Table of addresses A

Name	Address
Alice	Broad street 3
Ben	Broad street 3
Chris	Long street 5

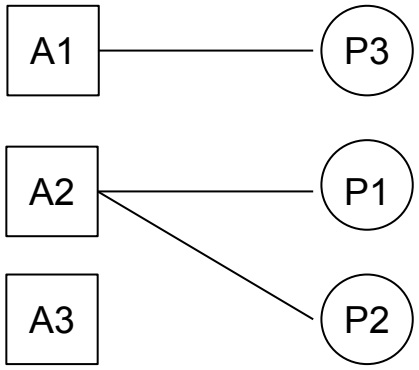
$L \subseteq P \times A$

	P1	P2	P3
A1	0	0	1
A2	1	1	0
A3	0	0	0

Adjacency matrix

A1: [P3]
A2: [P1, P2]
A3: []

Adjacency list



Visualization

Visual Channels

Visual channels and their effectiveness for representing data

Magnitude Channels: Ordered Attributes

- Position on common scale
- Position on unaligned scale
- Length (1D size)
- Tilt/angle
- Area (2D size)
- Depth (3D position)
- Color luminance
- Color saturation
- Curvature
- Volume (3D size)

Same

Same

Identity Channels: Categorical Attributes

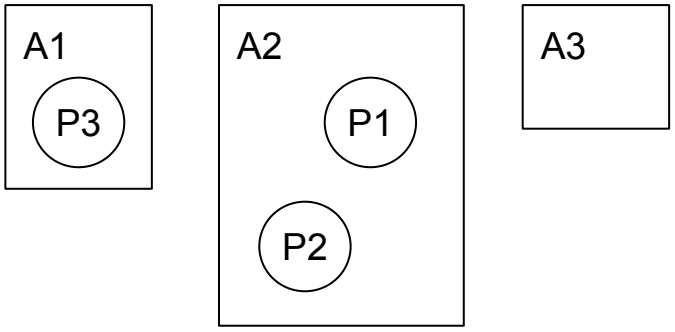
- Spatial region
- Color hue
- Motion
- Shape

Grouping Channels: Categorical Attributes

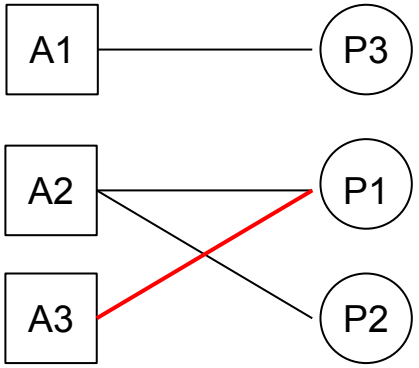
- Containment (2D)
- Connection
- Other identity channels

Source: Tamara Munzner, Visualization Design and Analysis, CRC Press, 2014.

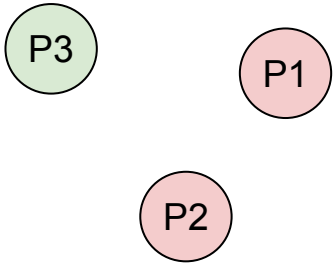
Visual encoding of relations



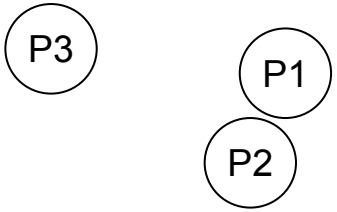
Visualization using containment
1:M binary relation
Not very scalable for N:M binary relations



Visualization using connections
1:M, N:M binary relations



Visualization using similarity (color hue)
1:M binary relation
Not very scalable



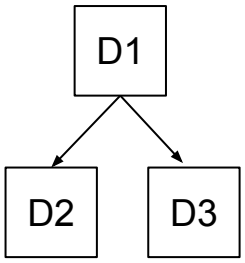
Visualization using proximity
1:M binary relation

Visual encoding of items

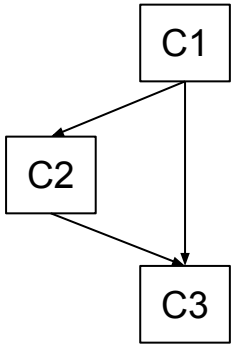
- We visually encode the items to geometry
- We can map attributes of the items on the properties of the geometry
  - Shape
  - Color
  - Size/area
- When we are using connections to visually communicate relations, we can map attributes of the links to properties of the connections
  - Width
  - Color

Examples of relational data

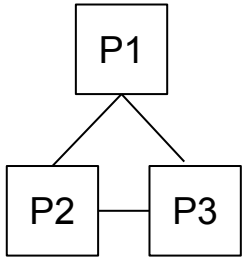
- File system - hierarchy
  - Directories are the items/nodes, parent of one dir in another dir (link)
- Prerequisites of courses - directed acyclic graph (DAG)
  - Courses are nodes, prerequisites of courses are the links
- Social network - undirected network
  - People are the items/nodes, friendships are the links
- World wide web - directed network
  - Pages are items/nodes, hyperlinks from one page to another are links



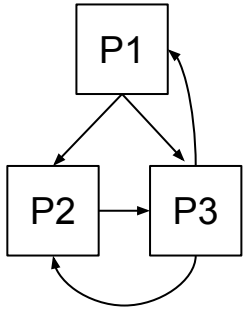
Hierarchy



DAG



Undirected network



Directed network

Example of hierarchy

Key	Name	Parent (FK)
D1	Home	
D2	Documents	D1
D3	Videos	D1

Table of directories D

Directory	Parent
Document	Home
Videos	Home

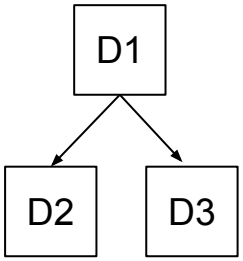
$L \subseteq D \times D$

	D1	D2	D3
D1	0	1	1
D2	0	0	0
D3	0	0	0

Adjacency matrix

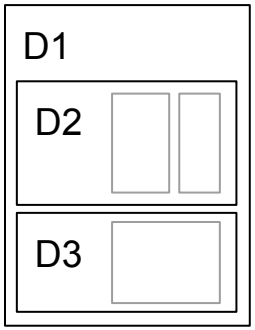
D1: [D2, D3]
D2: []
D3: []

Adjacency list

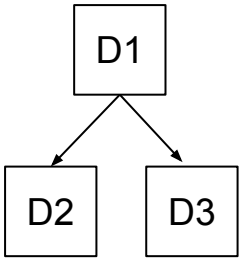


Visualization

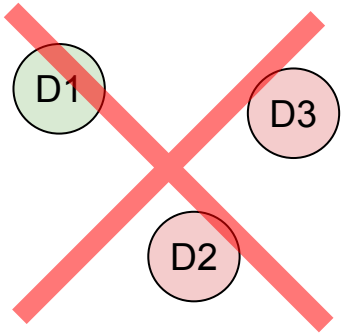
Visual encoding of hierarchies



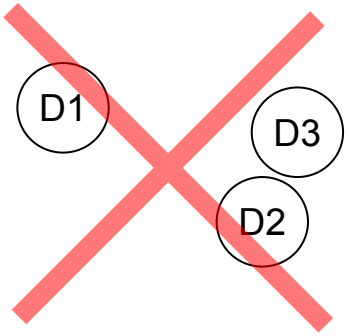
Visualization using containment
1:M binary relation



Visualization using connections
1:M binary relation



Visualization using similarity (color hue)
1:M binary relation
cannot express hierarchy



Visualization using proximity
1:M binary relation
cannot express hierarchy

Example of directed network
In this case DAG

Key	Name
C1	Algebra
C2	Graphics
C3	Visualization

Table of courses C

Key	Course (FK)	Required course (FK)
P1	C2	C1
P2	C3	C1
P3	C3	C2

Table of prerequisites P

Course	Required course
Graphics	Algebra
Visualization	Algebra
Visualization	Graphics

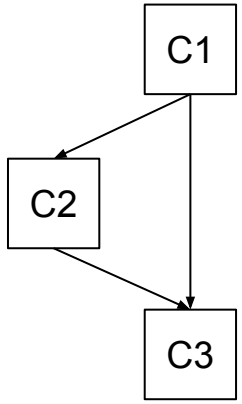
$L \subseteq C \times C$

	C1	C2	C3
C1	0	1	1
C2	0	0	1
C3	0	0	0

Adjacency matrix

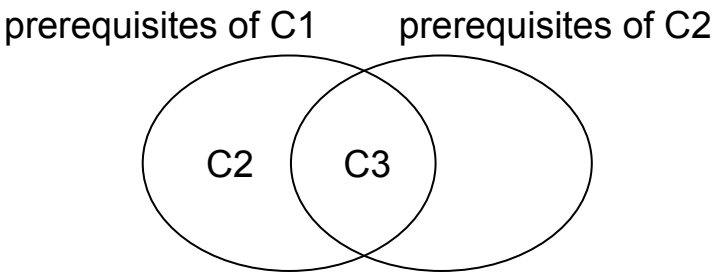
C1: [C2, C3]
C2: [C3]
C3: []

Adjacency list

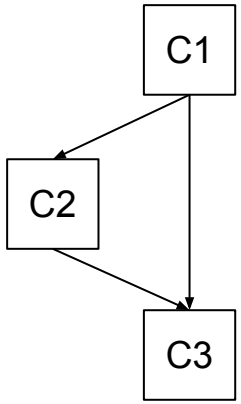


Visualization

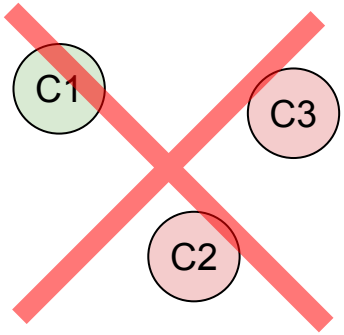
Visual encoding of directed network



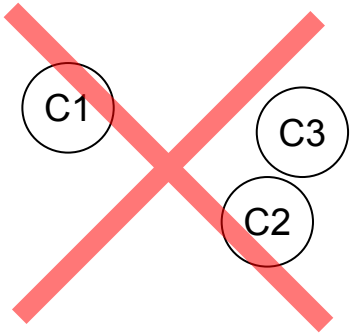
Visualization using containment
N:M binary relation
does not scale well



Visualization using connections
N:M binary relation



Visualization using similarity (color hue)
1:M binary relation
cannot express hierarchy
cannot express N:M binary relation



Visualization using proximity
1:M binary relation
cannot express hierarchy
cannot express N:M binary relation

Example of undirected network

Key	Name
P1	Alice
P2	Bob
P3	Carl

Table of people P

Key	Person1 (FK)	Person2 (FK)
F1	P1	P2
F2	P1	P3
F3	P2	P3

Table of friendship F

Person	Person
Alice	Bob
Alice	Carl
Bob	Carl

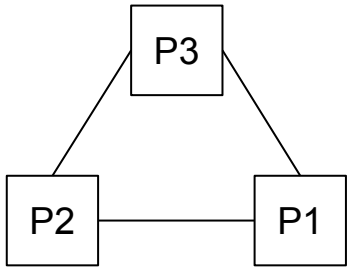
$L \subseteq P \times P$

	P1	P2	P3
P1	0	1	1
P2	1	0	1
P3	1	1	0

Adjacency matrix

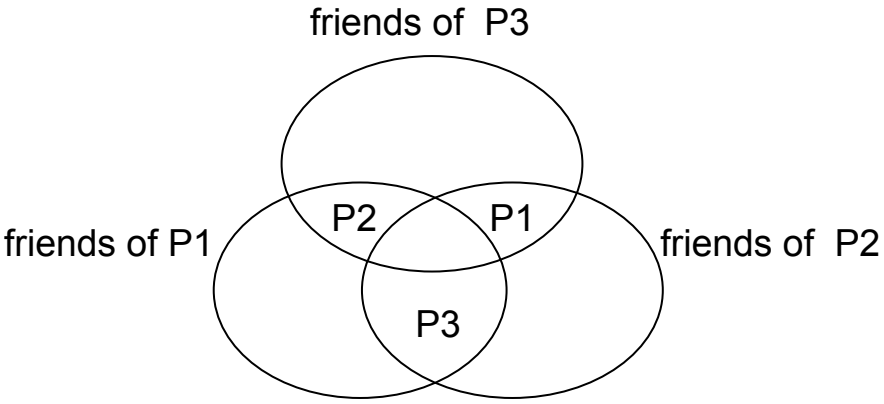
P1: [P2, P3]
P2: [P1, P3]
P3: [P1, P2]

Adjacency list

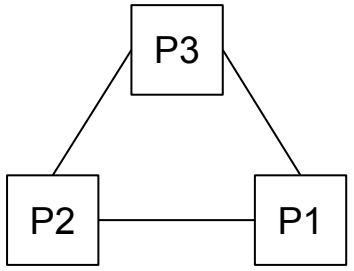


Visualization

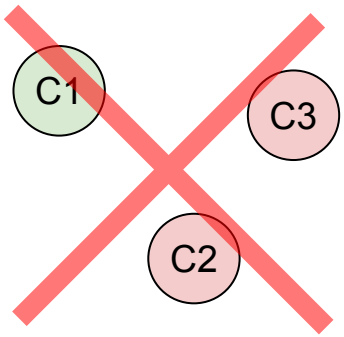
Visual encoding of undirected network



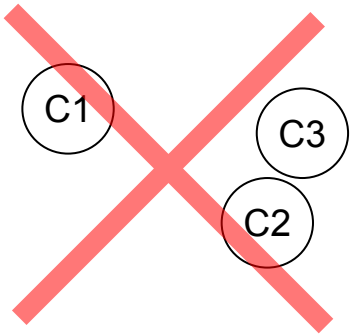
Visualization using containment
N:M binary relation
does not scale well



Visualization using connections
1:M, N:M binary relations



Visualization using similarity (color hue)
1:M binary relation
cannot express hierarchy
cannot express N:M binary relation



Visualization using proximity
1:M binary relation
cannot express hierarchy
cannot express N:M binary relation

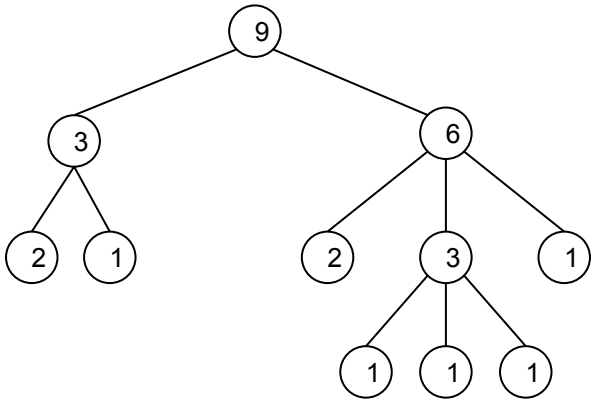
Tasks in visualization of relational data

- Identify task - target is one attribute
  - Visualization answers questions about value of individual attribute
  - What is value of attribute X for a given item?
- Identify task - targets are items
  - What items have value X of a given attribute?
  - What items have values of a given attribute in range [X, Y]?
  - What items are incident to a given link?
  - What items have more than X incident links?
- Identify task - targets are links
  - What links have value X of a given attribute?
  - What links have values of a given attribute in range [X, Y]?
  - What links are incident to given item?
- Identify task - targets are characteristics of the network
  - PATHS: What paths shorter that X links are between two given items?
  - COMPONENTS: How many and how big components are in a network?
  - LEVELS IN HIERARCHY: How many levels are there? How many items are on certain level?
  - etc.

Visualization of hierarchies with containment

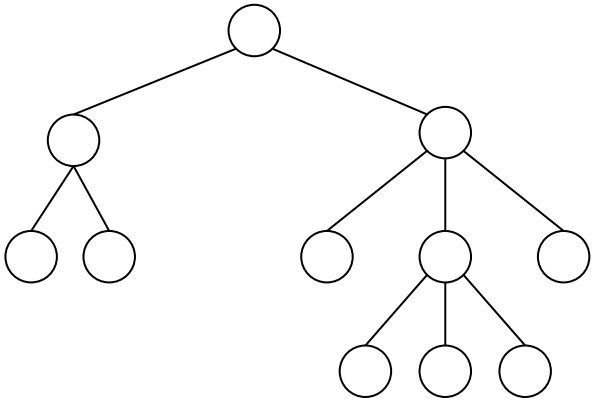
Treemap

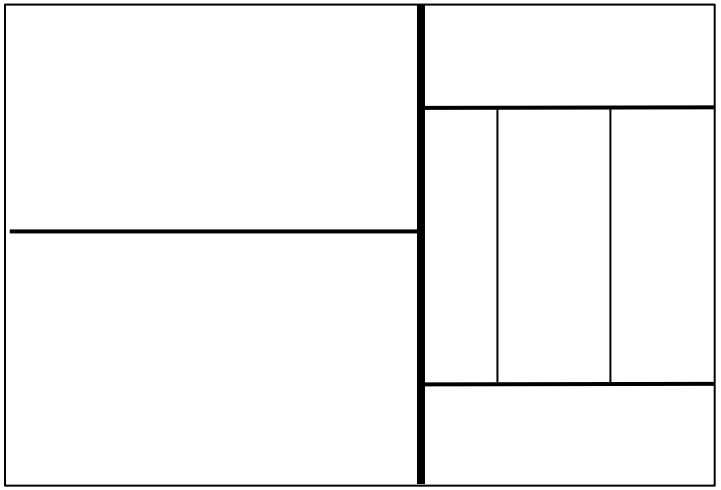
- Treemap is using containment to encode hierarchical relations
- It is mapping one attribute of the items in the hierarchy on size/area
  - It is expected that for inner nodes of the hierarchy, the value of the attribute is sum of the values of the attribute of all leafs of subtree of the node
- Treemap gives us overview of the whole hierarchy
  - More space is given to items with high value of the attribute that is mapped on size/area
- Treemap is useful for task where
  - We need to identify items of the hierarchy with certain value of one attribute
    - E.g., What directories are large



Treemap algorithm

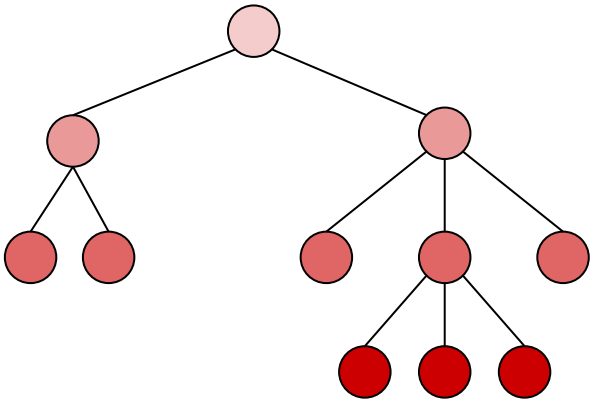
- Treemap is space dividing technique
- The dividing axis is axis X, starting node is root of the hierarchy
- We divide the rectangle representing a node of the hierarchy into smaller rectangles representing the children of the root
  - The size of the rectangles is determined by the value of certain attribute
  - For inner nodes of the hierarchy, the value of the attribute is sum of the values of the attribute of all leafs of subtree of the node
- We switch the dividing axis
- We continue to divide the new rectangles in depth-first or breadth-first order until there is no rectangle to divide (until there is no node with children)

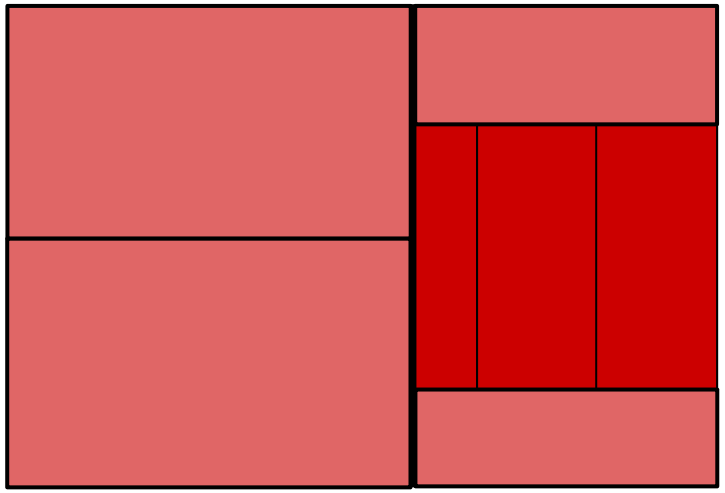




Treemap

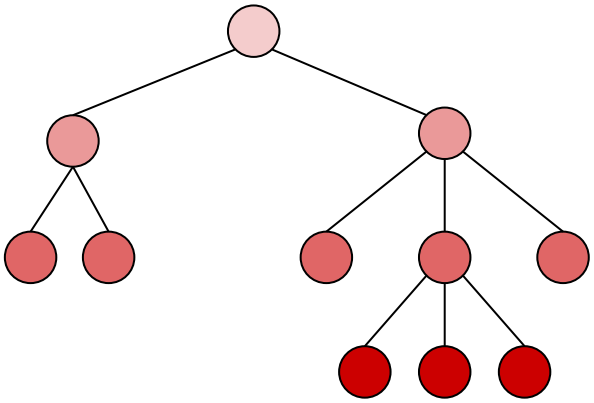
- Here we indicate the containment with the width of the dividing lines
- Other Attribute of the items and/or properties of the tree can be mapped on color of the rectangles
  - Here the level of the hierarchy the node is on is mapped on color

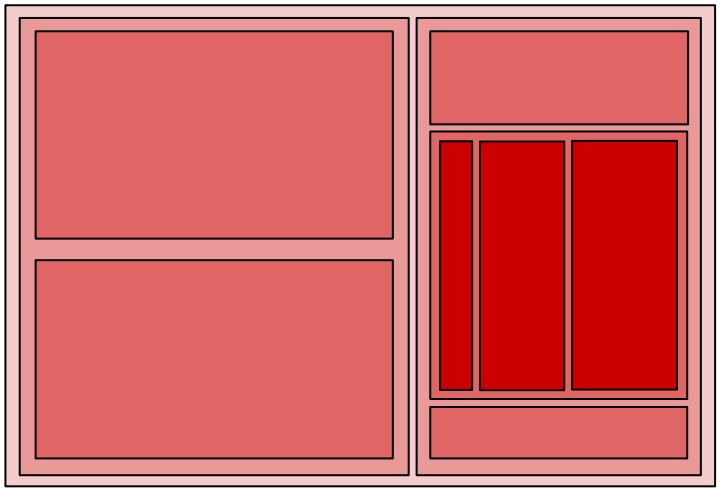




Treemap

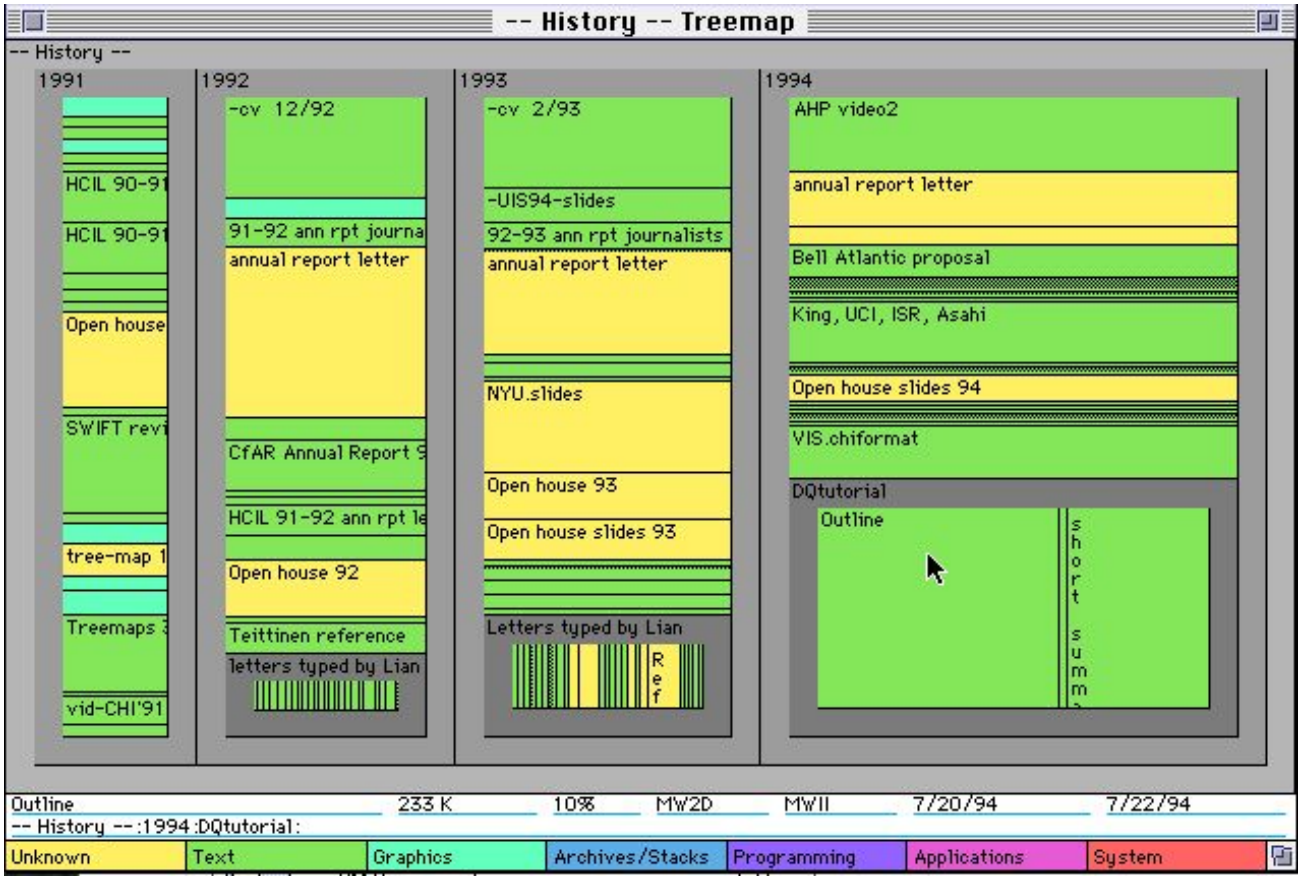
- Here the containment is really containment
- Other Attribute of the items and/or properties of the tree can be mapped on color of the rectangles
  - Here the level of the hierarchy the node is on is mapped on color





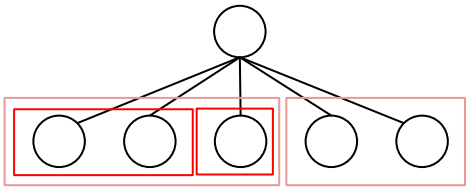
Example of a treemap
History of tasks and subtasks

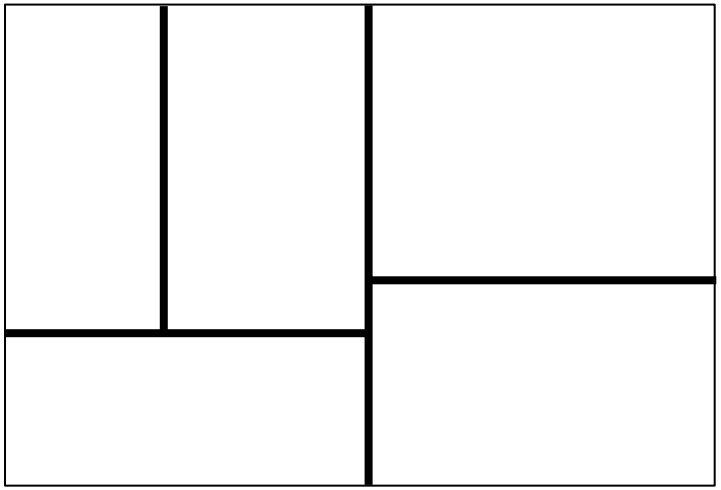
- Time needed to accomplish the task mapped on size
- Task type is mapped on color hue
- It allows us to see overview of the whole history
- We see which files and directories are big and which are small
- One drawback of this approach is that it generates lot of narrow rectangles



Improving the treemap

- There is simple approach how we can get rid of the narrow rectangles
- When we divide the rectangle representing certain node
  - Let us consider that the dividing axis is X axis
  - We divide the child nodes into two groups such that the groups have similar sum of the values of the attribute mapped on size of the rectangles
  - We divide the rectangle in two rectangles of a similar size
  - We switch the dividing axis
  - We recursively divide the new rectangles based on the remaining child nodes





Example of a treemap
Financial Market

S&P 500 Map - https://finviz.com/map.ashx?t=sec

Summary - treemap

- Treemap is using containment to encode hierarchical relations
- Tree map is space dividing technique
- It is mapping one attribute of the items in the hierarchy on size/area
  - It is expected that for inner nodes of the hierarchy, the value of the attribute is sum of the values of the attribute of all leafs of subtree of the node
- Treemap gives us overview of the whole hierarchy
  - More space is given to items with high value of the attribute that is mapped on size/area
- Treemap is useful for task where
  - We need to identify items of the hierarchy with certain value of one attribute
    - E.g., What directories are large
- Naive division of the space leads to narrow rectangles
  - We can generate fewer narrow rectangles with more sophisticated methods

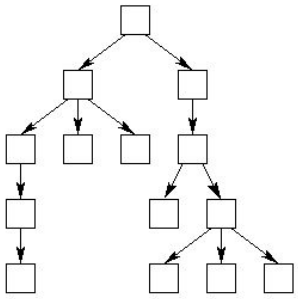
Visualization of hierarchies and networks with graphs

Visualization of networks

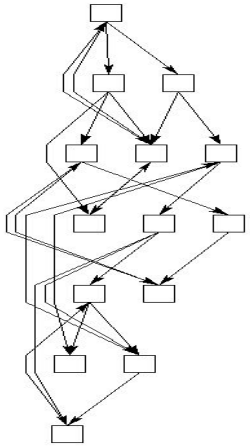
- Hierarchies and networks (DAG, directed, undirected) can be represented as graphs
  - With graphs we represent the items as nodes and the links as edges
  - 2D positions of the nodes are determined based on the structure of the network
    - We are calculating the **layout** of the graph
  - We can map attributes of the items to properties of the nodes
    - Shape
    - Color
    - Size
  - We can map attributes of the relations to properties of the edges
    - Width
    - Color

Layouts of graphs

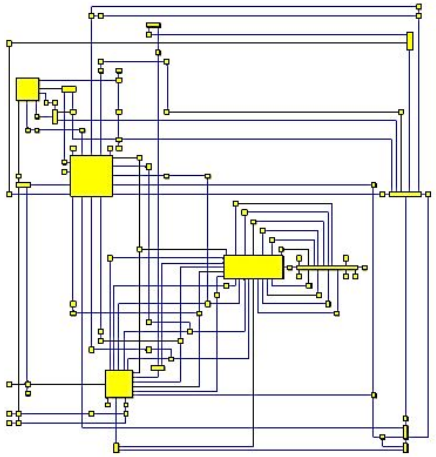
- There is bewildering range of various graph layouts and their variants
- The layouts are defined by
  - How the nodes are distributed in space (typically 2D space)
  - How the nodes are connected with edges



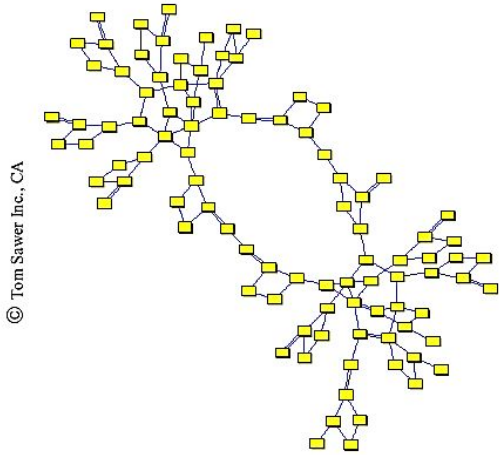
Tree layout



Hierarchical layout



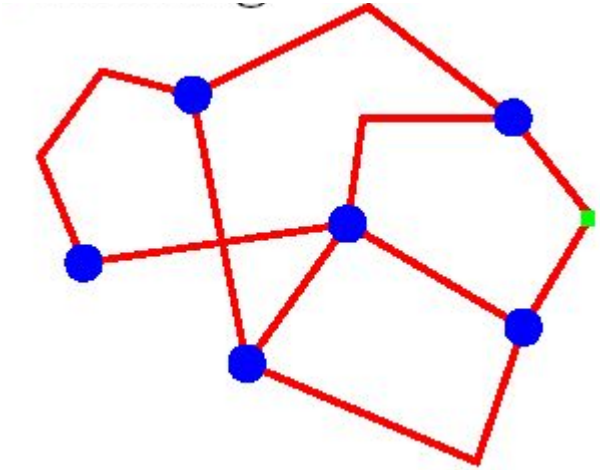
Orthogonal layout



Force-directed layout

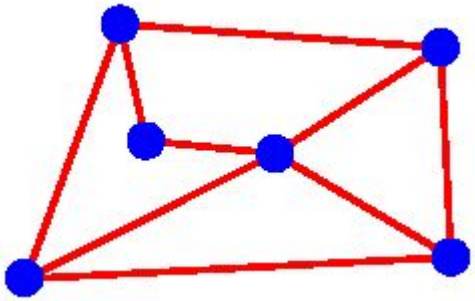
Layouts of graphs

- Various edges types

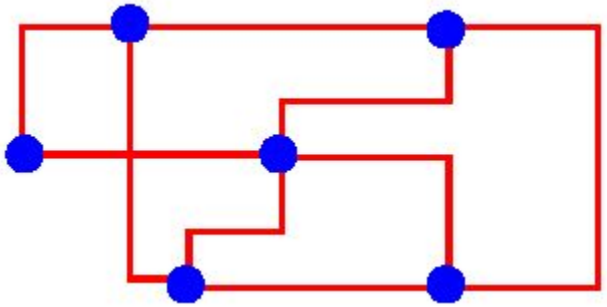


poly-line edges

can be used as control polygons for curves



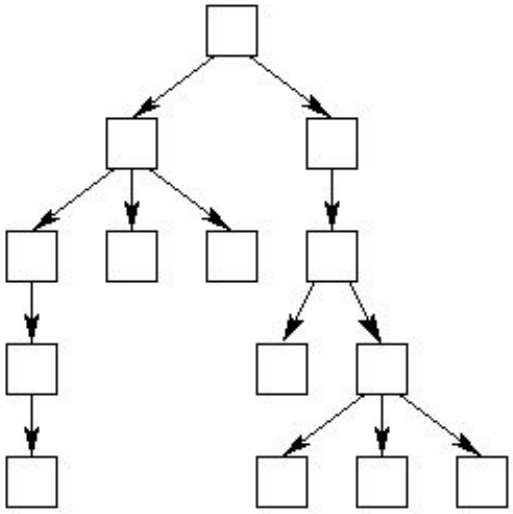
straight-line edges



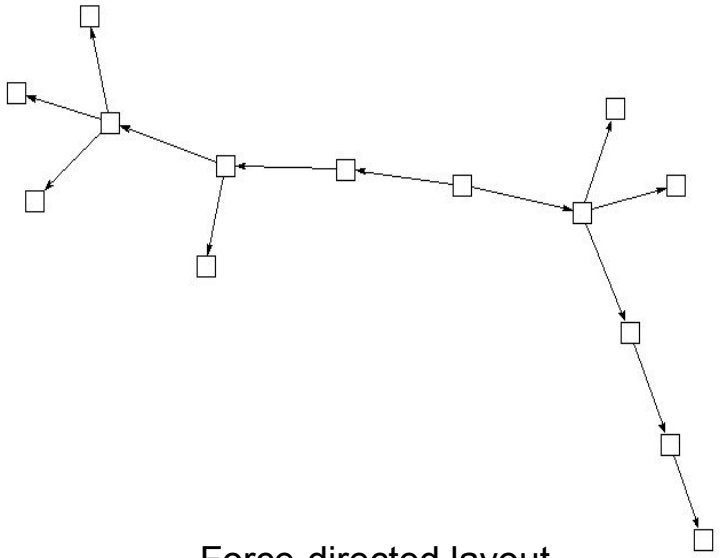
orthogonal edges

Various layouts are useful for various tasks

- Tree layout and hierarchical layout communicate the hierarchy much better than force-directed layout



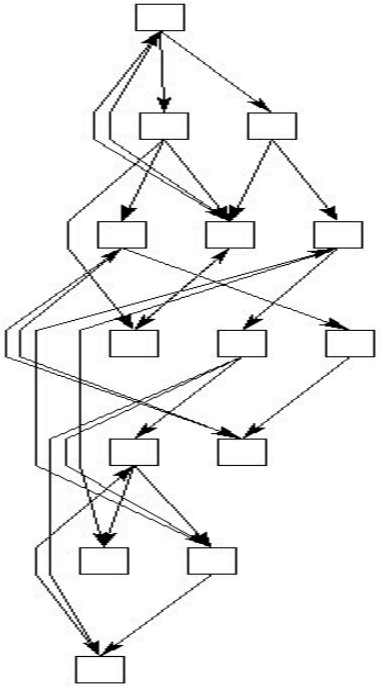
Tree layout



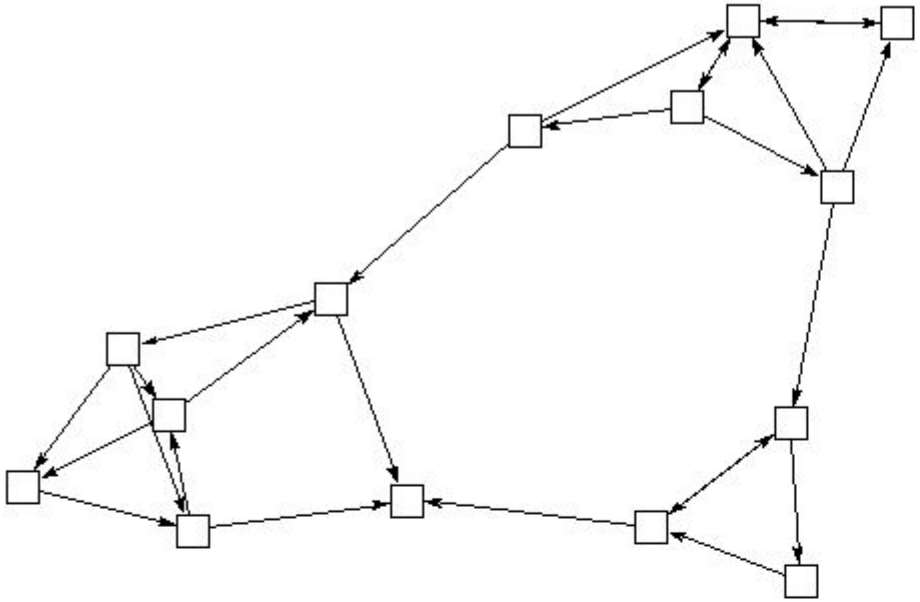
Force-directed layout

Various layouts are useful for various tasks

- Tree layout and hierarchical layout communicate the hierarchy much better than force-directed layout

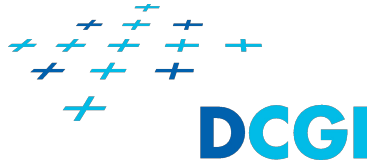


Hierarchical layout



Force-directed layout

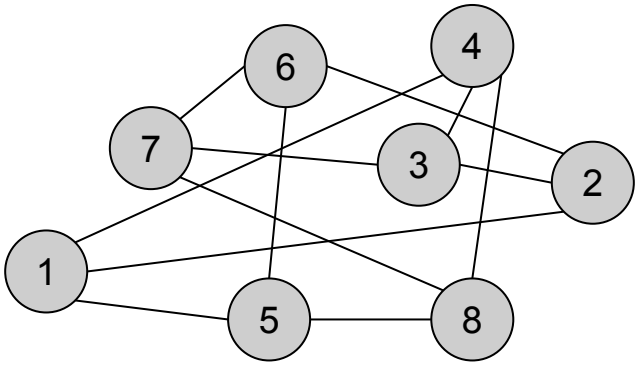
Aesthetics of graph layout



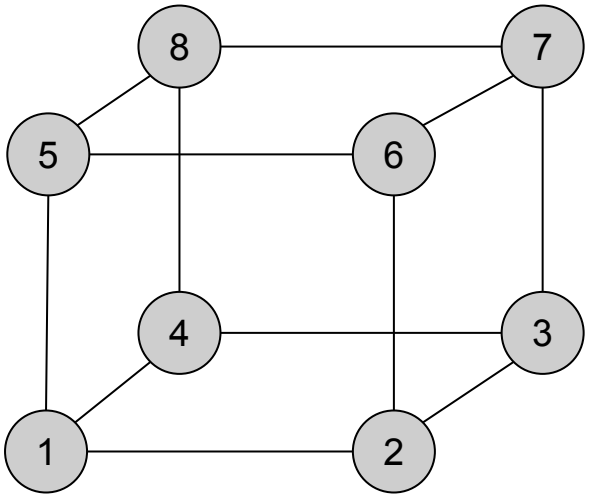
- The graph should exhibit certain aesthetics regardless of layout
- Aesthetics specify criteria we would want the graph to exhibit as much as possible
  - **Crossings** - minimization of the total number edge of crossings.
  - **Area** - minimize the graph area to save screen space
  - **Aspect Ratio** - minimization of the aspect ratio of the graph area
  - **Angular resolution** - maximization of the smallest angle between two edges incident on the same vertex
  - **Edge Length**
    - Total - minimization of the sum of the lengths of the edges
    - Maximum - minimization of the maximum length of an edge
    - Uniform - minimization of the variance of the lengths of the edges
  - **Bends**
    - Total - minimization of the total number of bends along the edges
    - Maximum - minimization of the maximum number of bends on an edge
    - Uniform - minimization of variance of the number of bends on an edge
  - **Symmetry** - display the symmetries of the graph in the drawing

Crossings

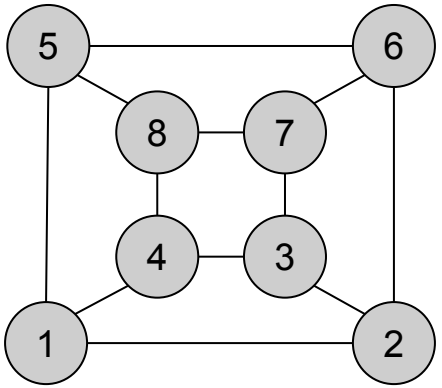
- Minimization of the total number edge of crossings
  - Ideally, we would like to have planar graphs (not always possible)



Random layout
12 edge crossings



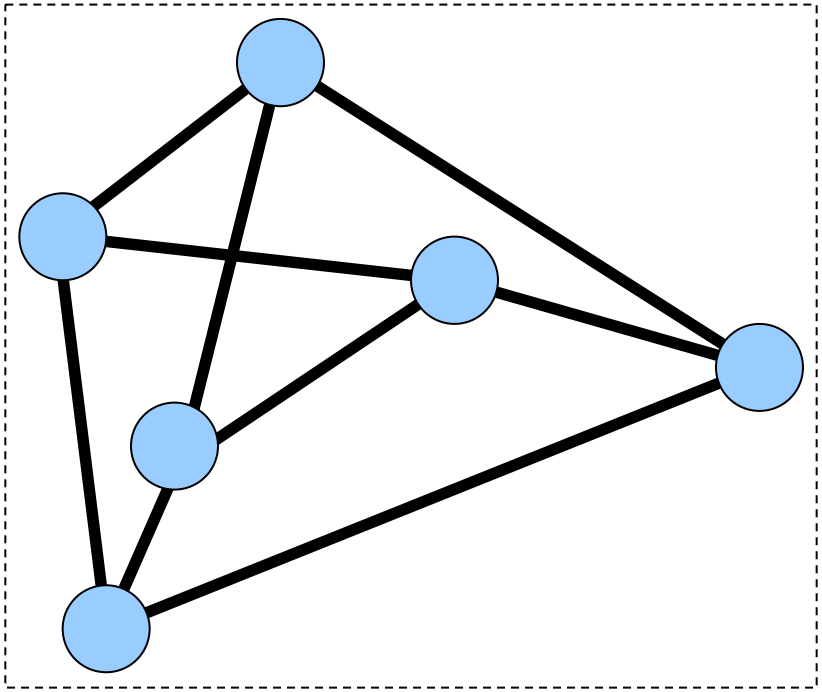
2 edge crossings



planar graph
0 edge crossings

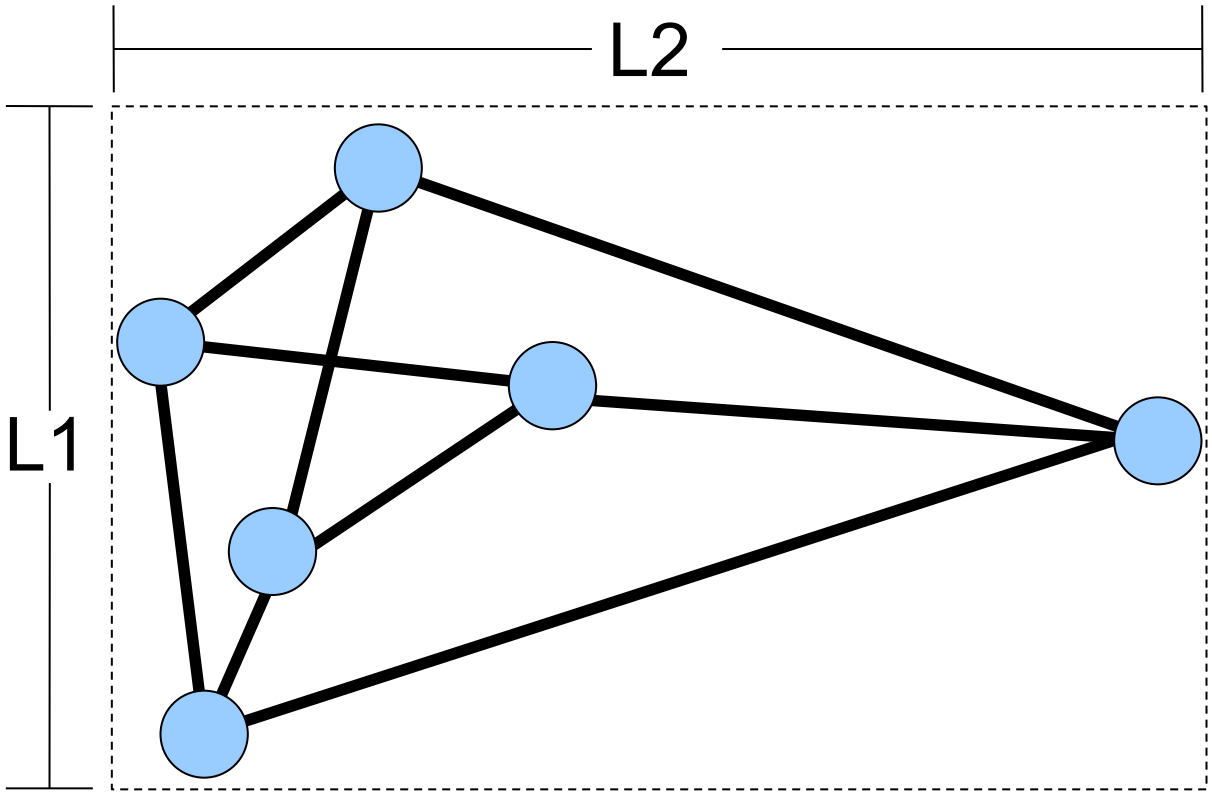
Area

- Minimization of the area of the graph
  - Important to save screen space



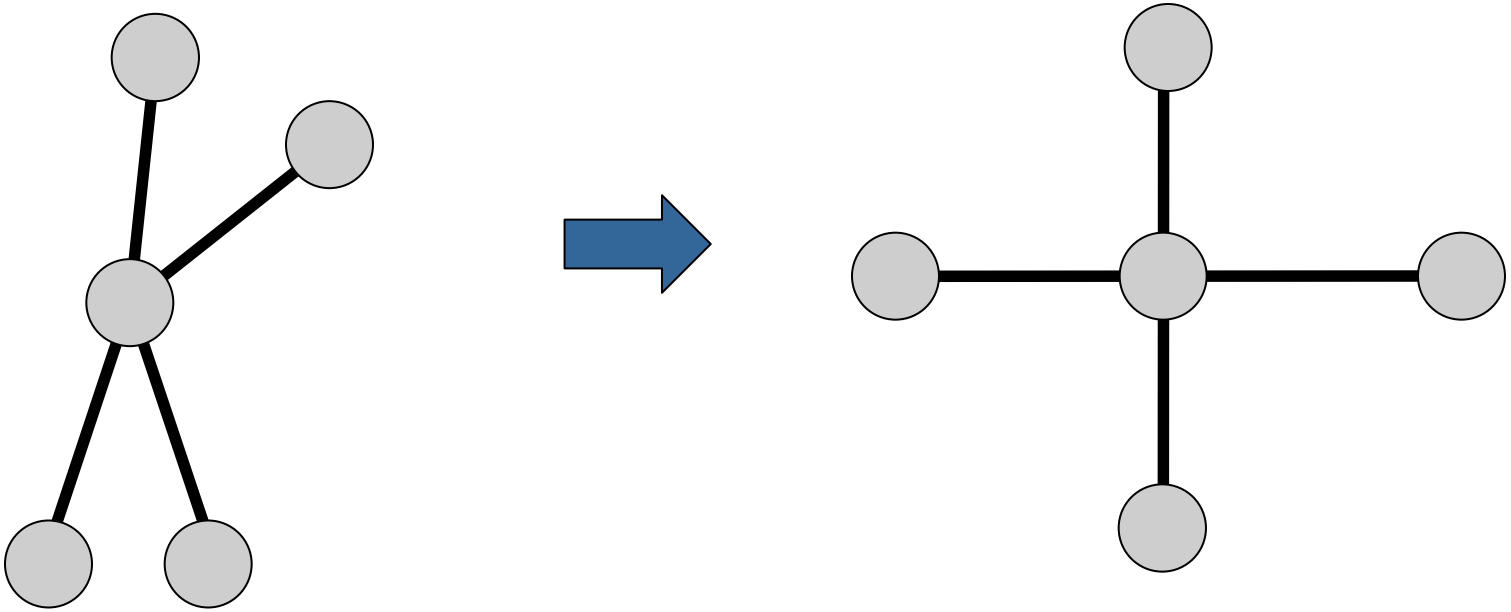
Aspect Ratio

- Minimization of the aspect ratio of the graph area
  - A.R. = L2/L1



Angular resolution

- Maximization of the smallest angle between two edges incident on the same vertex
  - This property helps also with aspect ratio and size of the graph area

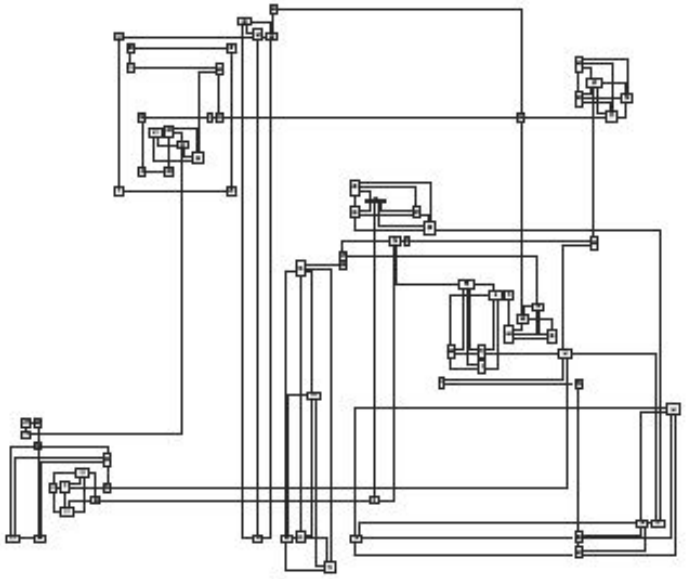


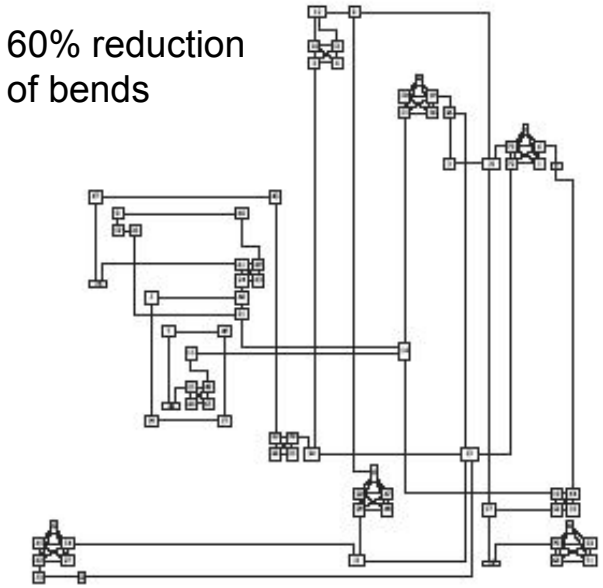
Edge Length

- Total Edge Length
  - Minimization of the sum of the lengths of the edges
- Maximum Edge Length
  - Minimization of the maximum length of an edge
- Uniform Edge Length
  - Minimization of the variance of the lengths of the edges
- Helps to make the graph area smaller
- Neighbour nodes will be close together

Bends

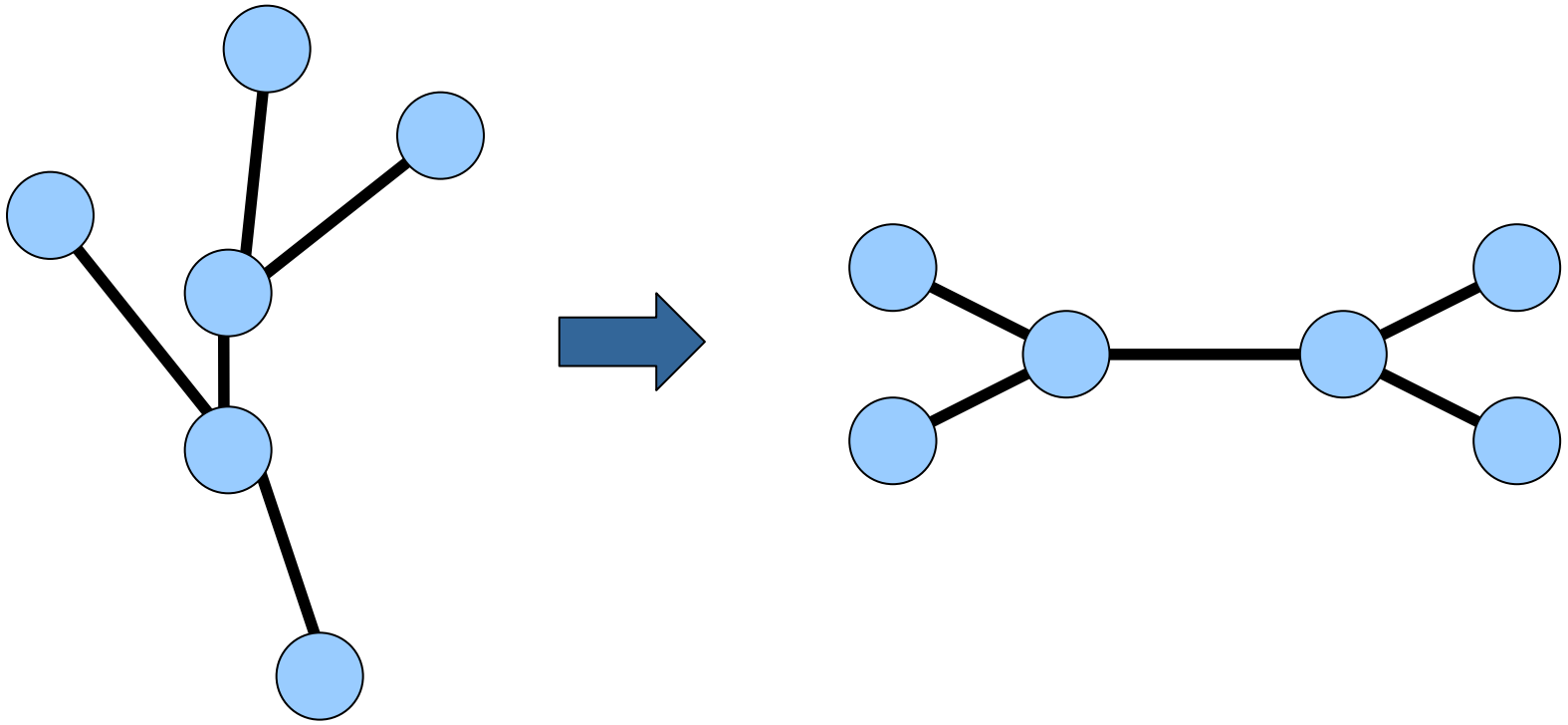
- Total Bends
  - Minimization of the total number of bends along the edges
- Maximum Bends
  - Minimization of the maximum number of bends on an edge
- Uniform Bends
  - Minimization of variance of the number of bends on an edge
- Important especially for orthogonal drawings
- Trivially satisfied by straight-line drawings
  - Often at the cost of more edge crossings





Symmetry

- Display the symmetries of the graph in the drawing

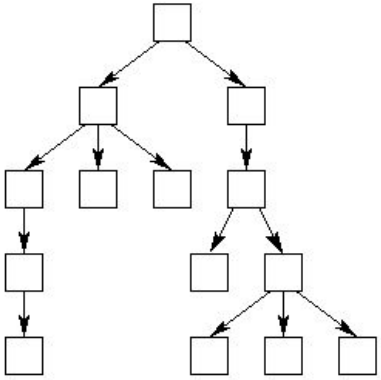


Summary - aesthetics of graph layout

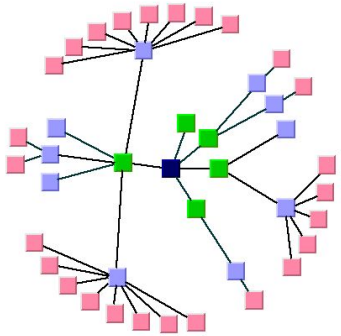
- The criteria are often contradicting
  - E.g., if we will minimize length of edges, we can make some angles between the edges small
  - We are searching for a balance between the criteria
- The search for the balance is an optimization problem
  - Multi-criteria optimization
  - Most of them are computationally hard.
- To speed up the computation of the graph layout we often ignore certain criteria and use approximation strategies and heuristics (e.g., gradient descent)
  - The heuristics will typically find local minimum only

Visualization of hierarchies with graphs

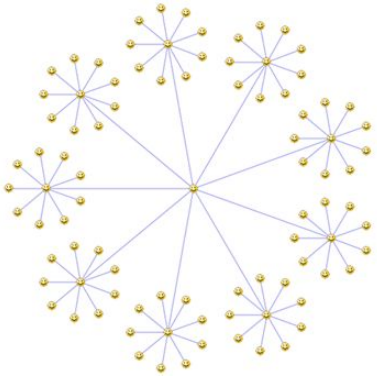
- Again, there is bewildering range of tree layouts
- We will show properties only of several tree layouts
- We will show algorithm only for hierarchical tree layout



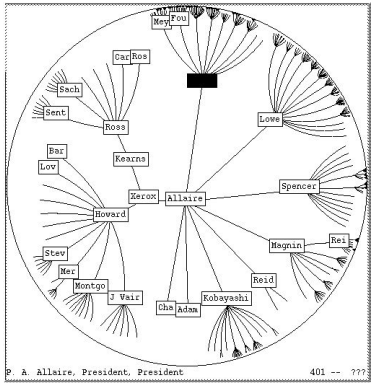
Hierarchical tree layout



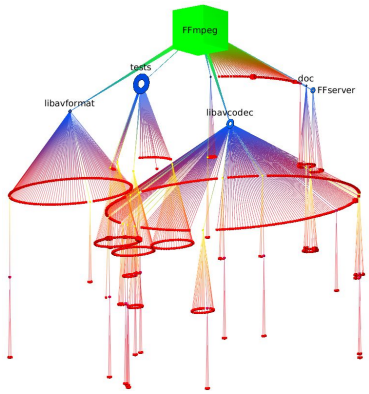
Radial tree



Balloon tree



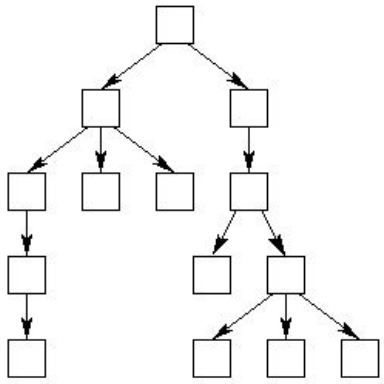
Hyperbolic tree



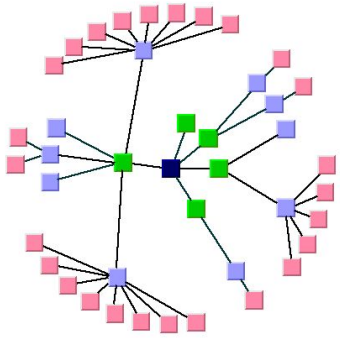
Cone tree

Visualization of hierarchies with graphs

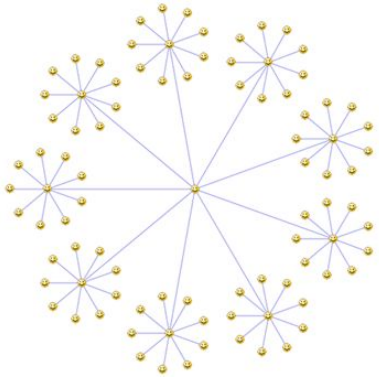
- All layouts communicate hierarchy of the graph
- Hierarchical tree layout
  - If there are many leaves and many levels, then the graph can be both wide and tall
  - Scrolling or zoom will be required
- Radial tree
  - Perimeter of a circle is longer than the width of screen
  - We can show only limited number of hierarchy levels
  - User can change which node will be in center of the circle
- Balloon tree
  - Subtree is always in proximity of its root
  - User can zoom to see only one subtree



Hierarchical tree layout



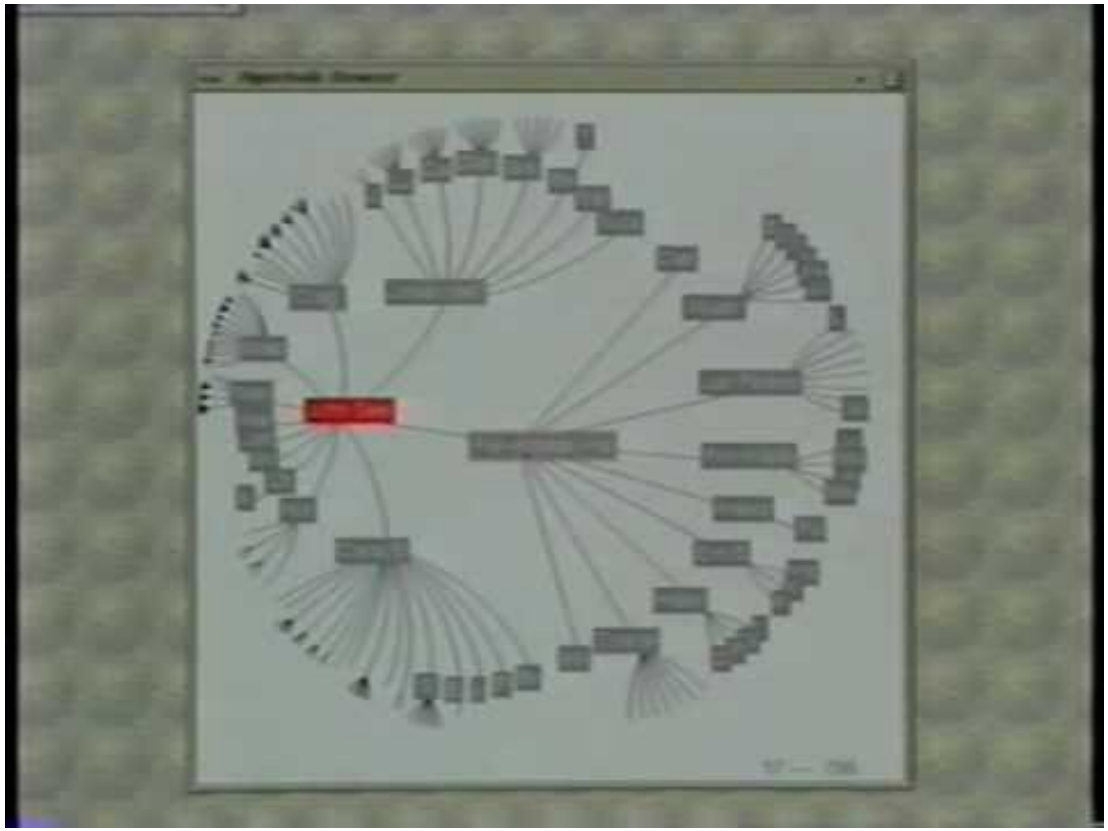
Radial tree



Balloon tree

Hyperbolic tree

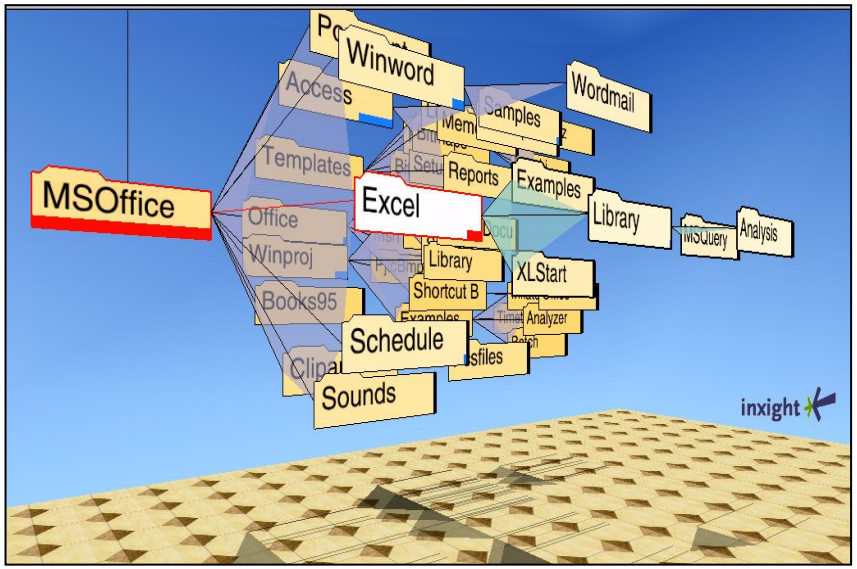
- Variant of radial tree
- Uses hyperbolic space
  - We have more space for nodes in the center of the hyperbolic space
- Focus + context technique



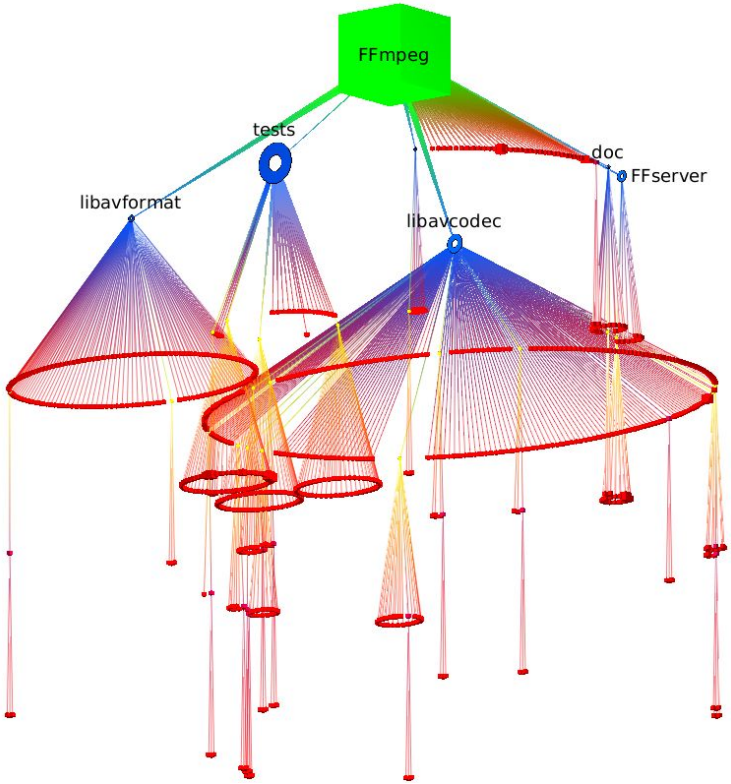
https://www.youtube.com/watch?v=8bhq08BQLDs

Cone Tree

- 3D variant of the balloon tree
- Useful when nodes have large number of children

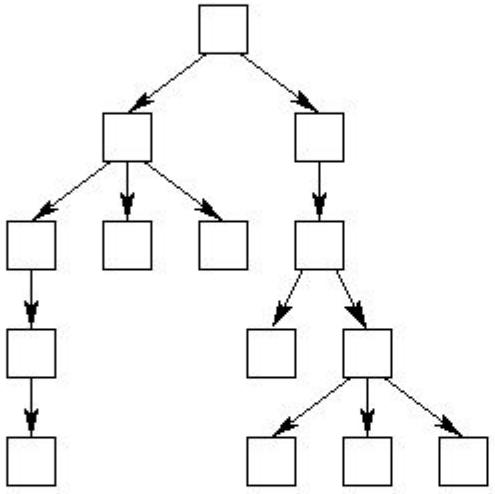


Cone tree – Robertson, Mackinlay, and Card

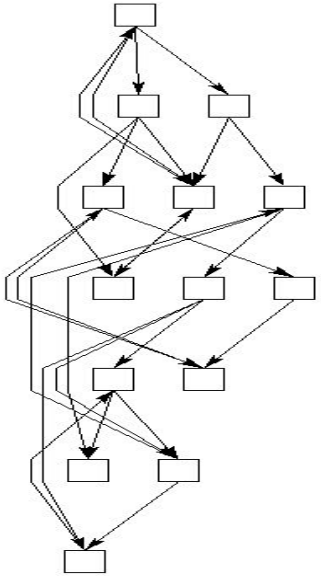


Sugiyama framework

- An algorithm for hierarchical layout of trees or directed acyclic graphs (DAGs)
- If the input graph is not DAG, we convert it to DAG first
- For tree, the result will be hierarchical tree layout
- For DAG, the result will be hierarchical layout



Hierarchical tree layout



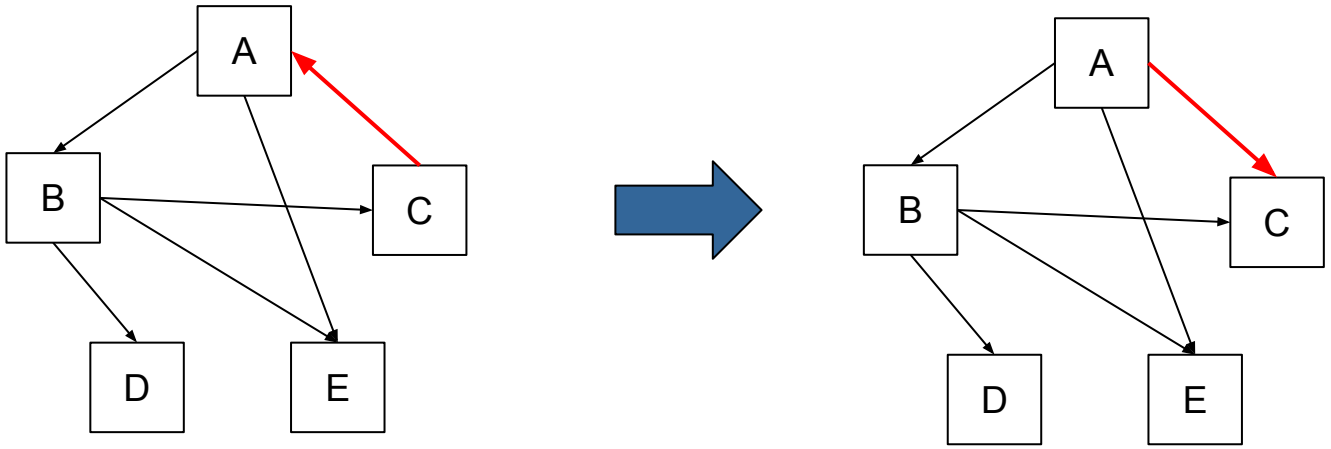
Hierarchical layout

Sugiyama framework

1. Remove cyclecles
   - If the graph is not DAG, switch one edge in each cycle to remove the cycles
2. Assign layers to nodes
   - Every node is assigned to a layer based on topological ordering of the graph - each child node is in lower layer than its parents
3. Reduce long edges
   - If there is an edge between two nodes in non-adjacent layers (e.g., 1 and 4) then put virtual nodes into layers that are between the non-adjacent layers (e.g., layers 2 and 3)
4. Aggregate virtual nodes
   - Optional step, we can aggregate multiple virtual nodes to which the edges are going from the same node into one node
5. Reduce edge crossings
   - Optimization step, we are reducing the number of edge crossings by switching positions of the nodes
6. Assign coordinates
   - We assign coordinates to the nodes based on their size

Sugiyama framework

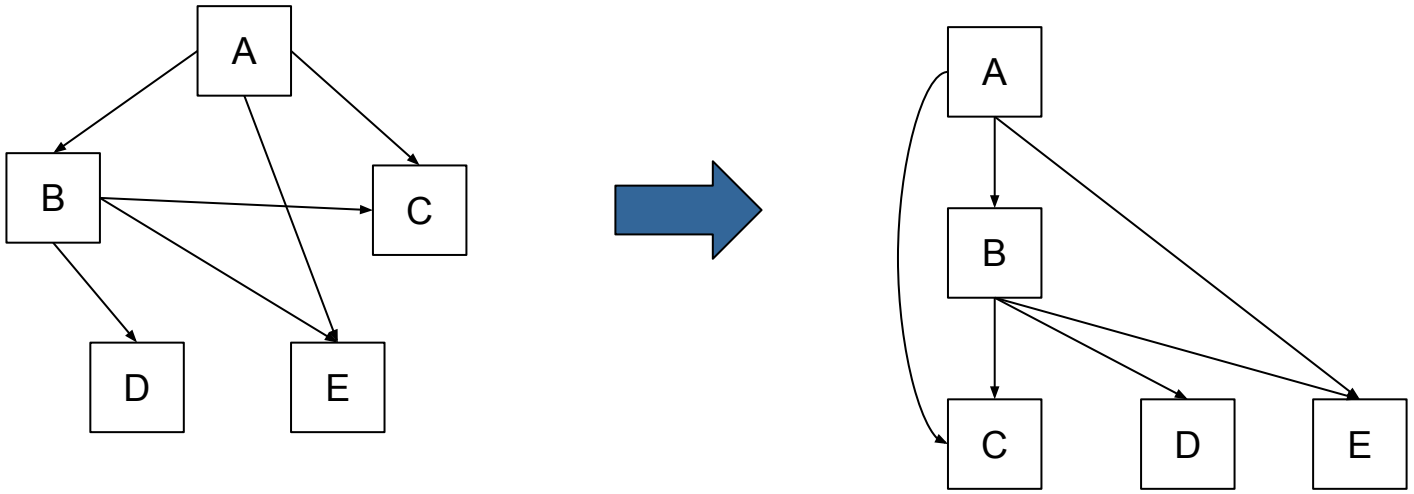
1. Remove cyclecles
   - If the graph is not DAG, switch one edge in each cycle to remove the cycles



Sugiyama framework

2. Assign layers to nodes

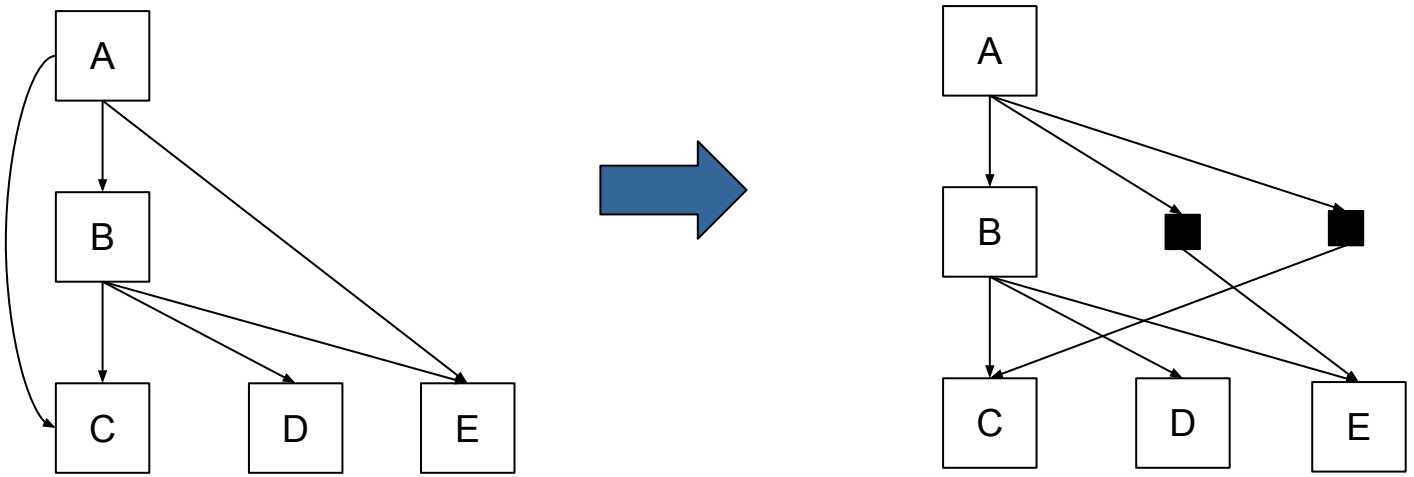
- Every node is assigned to a layer based on topological ordering of the graph - each child node is in lower layer than its parents



Sugiyama framework

3. Reduce long edges

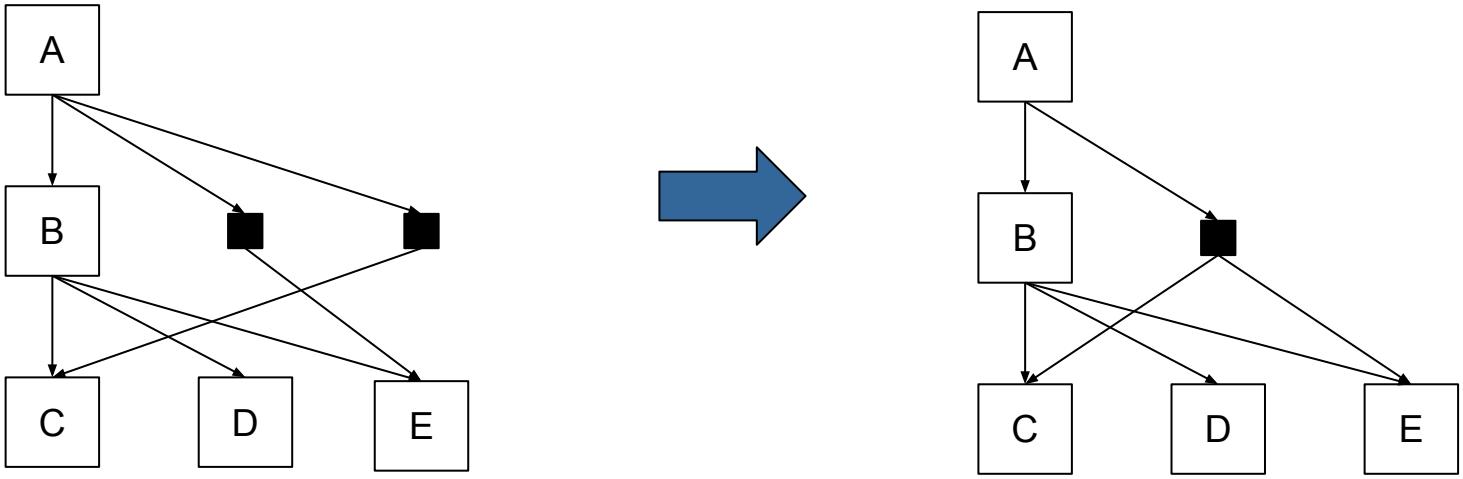
- If there is an edge between two nodes in non-adjacent layers (e.g., 1 and 4) then put virtual nodes into layers that are between the non-adjacent layers (e.g., layers 2 and 3)



Sugiyama framework

4. Aggregate virtual nodes

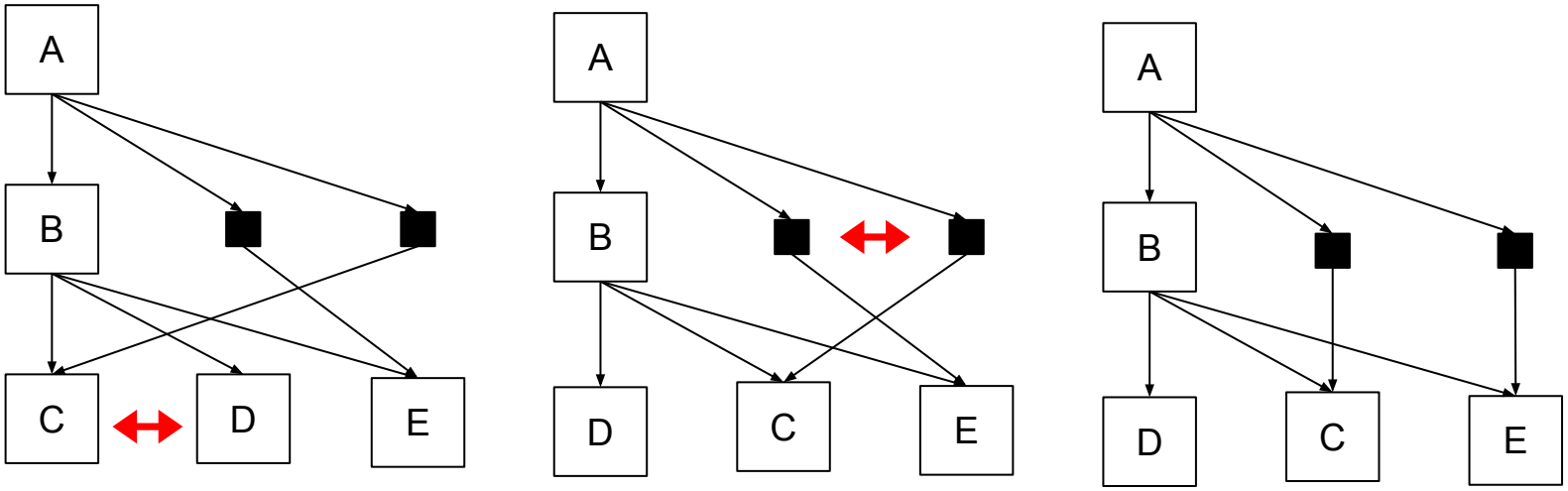
- Optional step, we can aggregate multiple virtual nodes to which the edges are going from the same node
- We will not aggregate the nodes as one of the edges is the edge with switched direction



Sugiyama framework

5. Reduce edge crossings

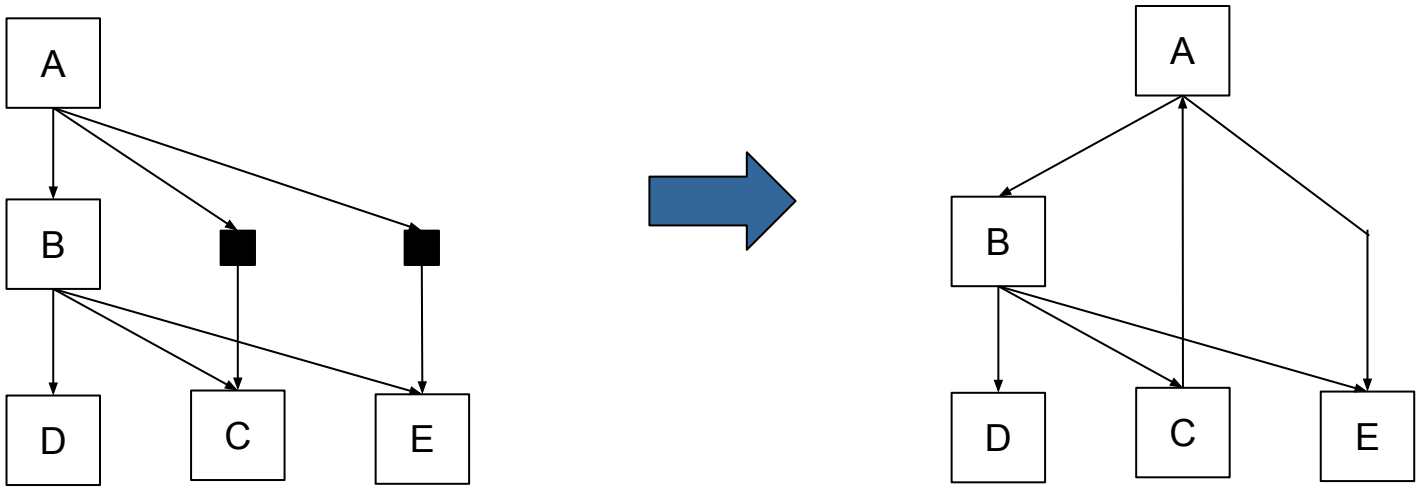
- Optimization step, we are reducing the number of edge crossings by switching positions of the nodes



Sugiyama framework

6. Assign coordinates

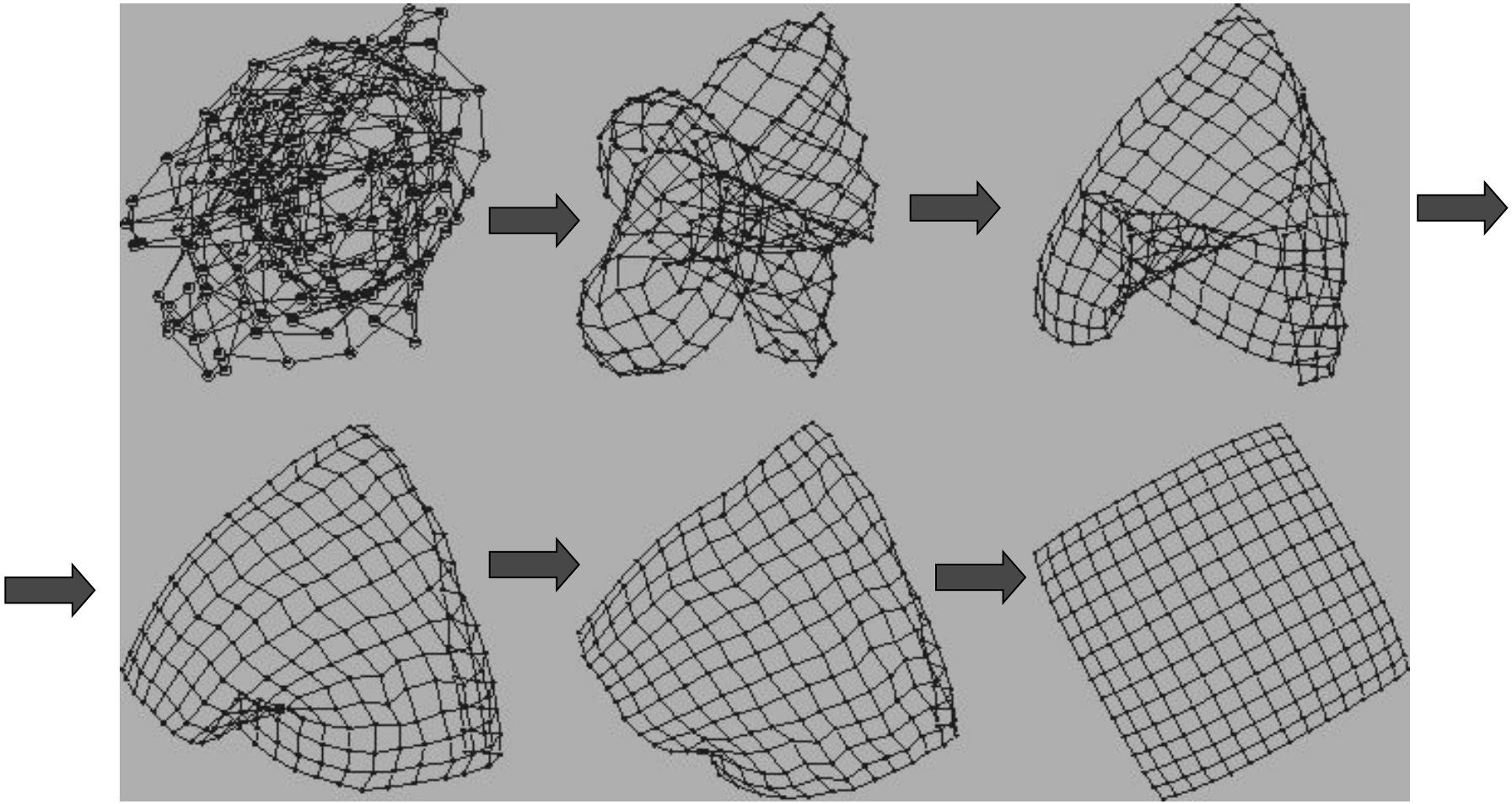
- We assign coordinates to the nodes based on their size and the number and size of their children
- We remove the virtual nodes
- We switch back direction of the edges that we switched in step 1



Force-directed methods

- Use a physical analogy to draw graphs
  - How does nature “draw” a graph?
- View a graph as a system of objects with forces acting between them.
- Assumption
  - A balanced system gives a good layout
- Specifically, a system configuration with locally minimal energy:
  - The sum of the forces on each object is zero
  - Such state is very often not global minimum but local minimum

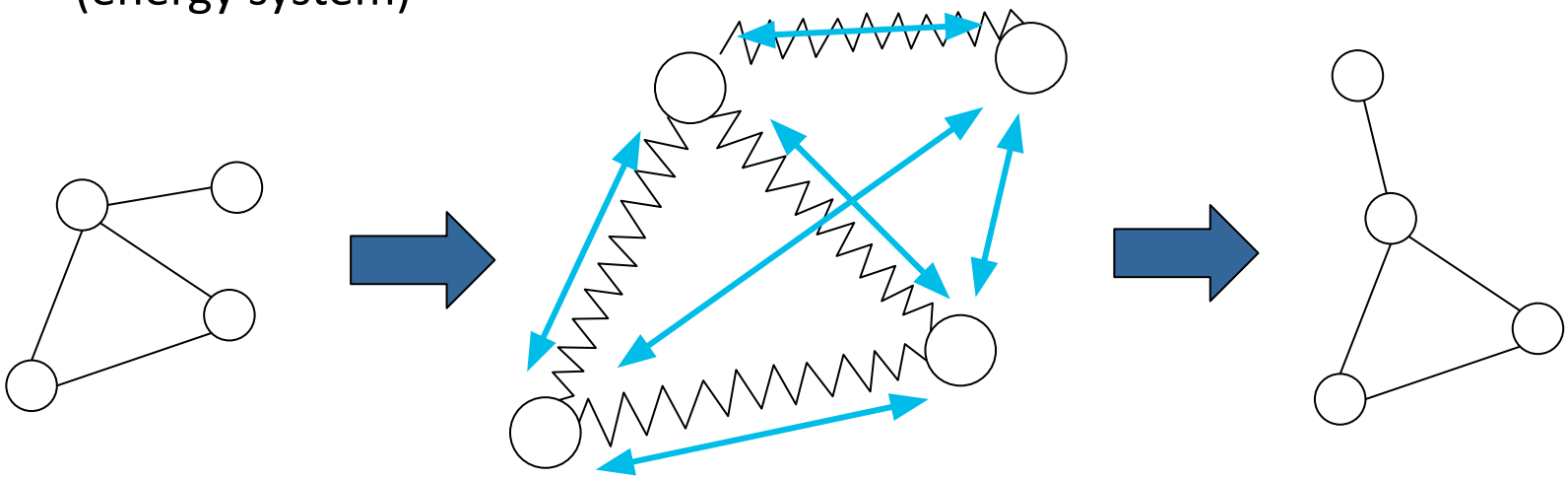
Example of force-directed layout algorithm



© Sander

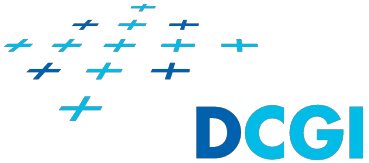
Force-directed methods

- There are many force directed methods.
- In general, they have two parts
  - **model**
  - **layout algorithm**
- The model
  - A force system defined by nodes and edges
- The layout algorithm
  - A technique for finding an equilibrium state, that is the sum of the forces on each vertex is zero (force system) or
  - A technique for finding a configuration with locally minimal energy (energy system)



Force-directed model

Eades, Peter (1984), "A Heuristic for Graph Drawing", Congressus Numerantium, 42 (11): 149–160.



- Nodes connected by edges will exert an attraction force:

$$F_a = c_1 \cdot log(d/c_2)$$

d is distance between the nodes

c_1 is the strength of attraction, recommended value is 2

c_2 is the ideal edge length, the attractive force will be 0 when distance of the vertices is equal c_2, recommended value is 1

- Nodes that are not connected with edge will exert a repelling force on each other

$$F_r = c_3/\sqrt{d}$$

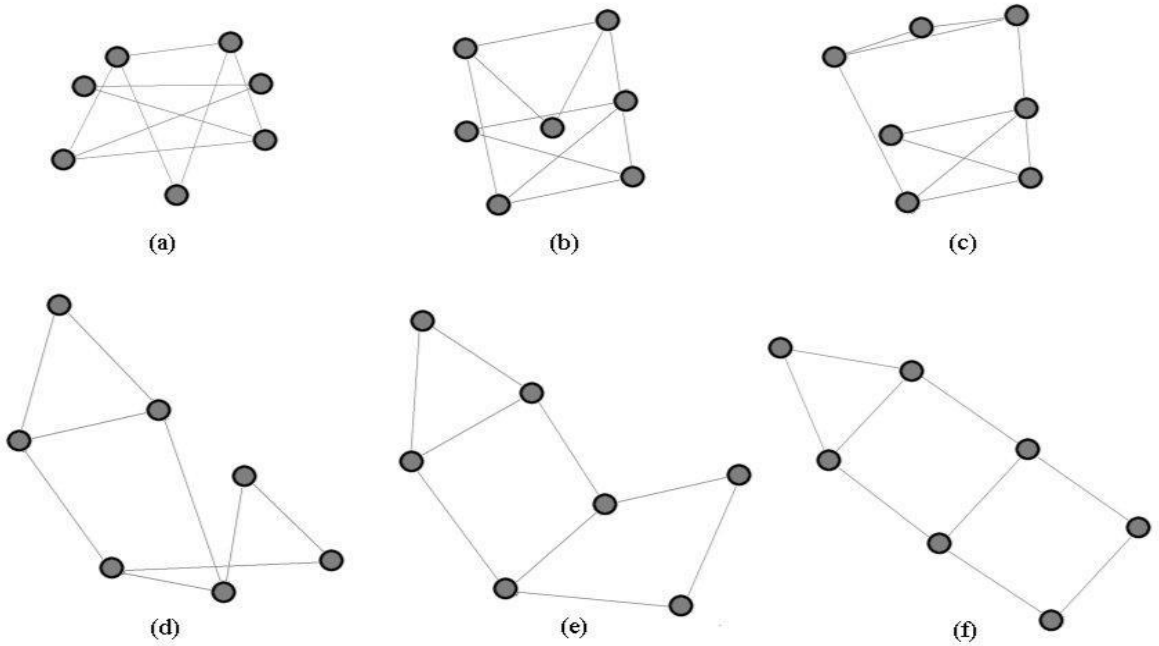
d is distance between the nodes

c_3 is the strength of repulsion, recommended value is 1

Force-directed layout algorithm

Eades, Peter (1984), "A Heuristic for Graph Drawing", Congressus Numerantium, 42 (11): 149–160.

- We start with pseudo-random positions of the nodes
- Repeat (recommended number of iterations is 100)
  - For each node, calculate the total force acting upon it
  - Move each node a small distance in the direction of the force acting upon it (add the force vector multiplied by constant $c_4 = 0.1$)



Force-directed model

Fruchterman, Thomas M. J.; Reingold, Edward M. (1991), "Graph Drawing by Force-Directed Placement", Software: Practice and Experience, 21 (11), Wiley: 1129–1164.

- The physical analogy is that every node in the graph is a charged electric particles and every edge is an elastic spring
- Ideal distance between two nodes is $k = C \cdot \sqrt{\frac{area}{|V|}}$
  - C is a scaling constant
- Nodes connected by edges will exert an attraction force:

$$F_a = d^2/k$$

- All nodes will exert a repelling force on each other, regardless of whether they are connected or not

$$F_r = k^2/d$$

- For each two nodes connected by edge there is an equilibrium
  - At distance $d = k$ where the attractive force and the repulsive force cancel each other out

Force-directed layout algorithm

Fruchterman, Thomas M. J.; Reingold, Edward M. (1991), "Graph Drawing by Force-Directed Placement", Software: Practice and Experience, 21 (11), Wiley: 1129–1164.

- Introduction of temperature that influence movement of the nodes
- We start with pseudo-random positions of the nodes
- We set the temperature to high
- Repeat
  - For each node, calculate the total force acting upon it
  - Move each node a distance in the direction of the force acting upon it based on the temperature
    - If a node would be moved out of a boundary (rectangle) it is stopped by the boundary
  - Decrease the temperature
- Stop when
  - The temperature is lower than given threshold or
  - The sum of the forces acting upon all nodes is lower than given threshold or
  - The maximum number of iterations is reached

Force-directed model

Kamada, Tomihisa; Kawai, Satoru (1989), "An algorithm for drawing general undirected graphs", Information Processing Letters, 31 (1), Elsevier: 7–15.

- We have springs between all pairs of nodes
  - Length of spring between two nodes is given by shortest path distance between the nodes
- Initially the nodes are placed at random positions
- Potential energy E stored in each spring is integral of force given by Hooks law

  $E = \frac{1}{2}kx^2$, k is stiffness of the spring, x is displacement of the spring
- **Euclidean distance** between two nodes on the screen should be proportional to the **shortest path distance** between them in the graph.
- Minimizes global potential energy of all springs

$$E = \sum_{i<j} \frac{1}{2} k_{ij} \left(|p_i - p_j| - l_{ij} \right)^2$$

  - $|p_i - p_j|$ is **Euclidean distance** between two nodes
  - $l_{ij} = L \cdot d_{ij}$ is the desired distance, L is constant, d_{ij} is **shortest path distance** between the nodes i and j
  - $k_{ij} = K / d_{ij}^2$ is the spring stiffness, K is constant, d_{ij} is **shortest path distance** between the nodes i and j

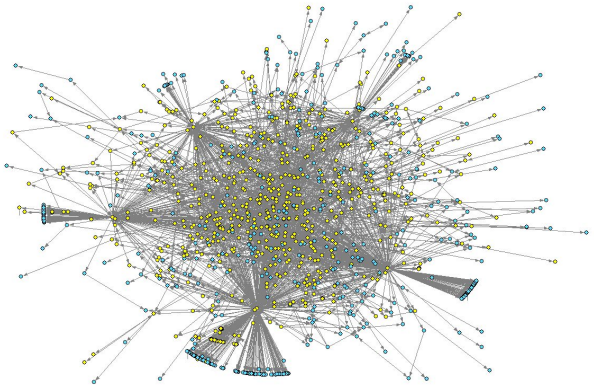
Force-directed layout algorithm

Kamada, Tomihisa; Kawai, Satoru (1989), "An algorithm for drawing general undirected graphs", Information Processing Letters, 31 (1), Elsevier: 7–15.

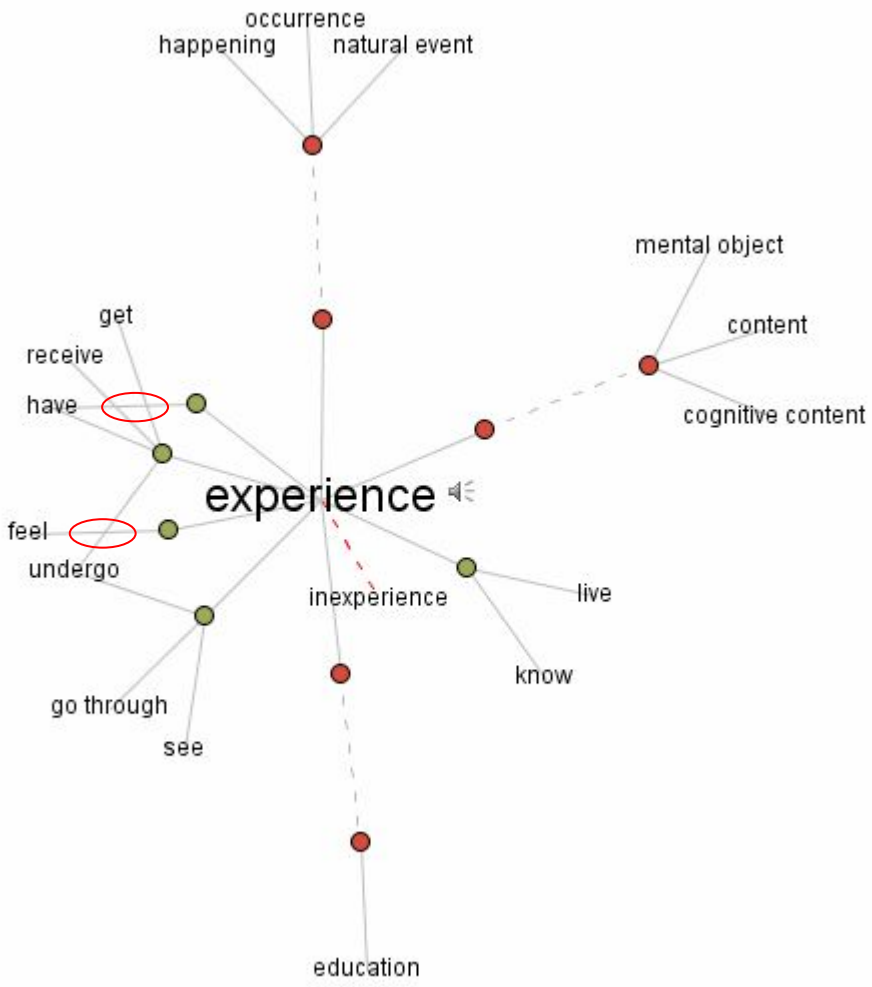
- We start with pseudo-random positions of the nodes
- Repeat
  - Find the node with max. potential energy
  - Iteratively move the node to equilibrium (using Newton-Raphson)
    - First derivative of the potential energy is the force acting upon the node. It gives us the direction in which we need to move.
    - Second derivative of the potential energy gives us the rate of change of the force acting upon the node. It says how far we need to jump.
    - We stop when the distance from the equilibrium is lower than given threshold
- Stop when
  - The maximum potential energy over all springs is lower than given threshold

Problems of force-based methods

- They do not consider edge crossings
- Hairball result for dense graphs
- The result is very often a local minimum



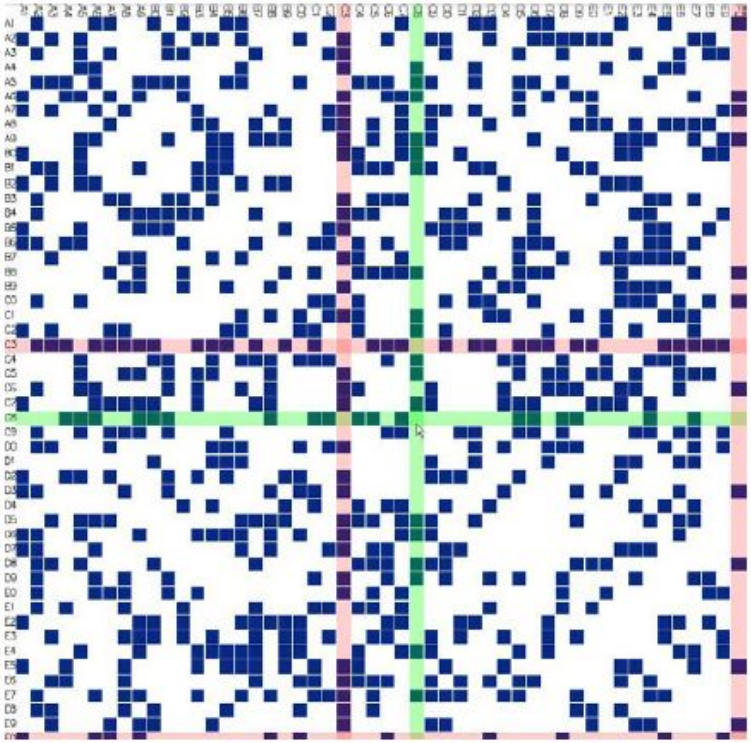
An example of hairball result



Source: www.visualthesaurus.com

Matrix-based visualization of networks

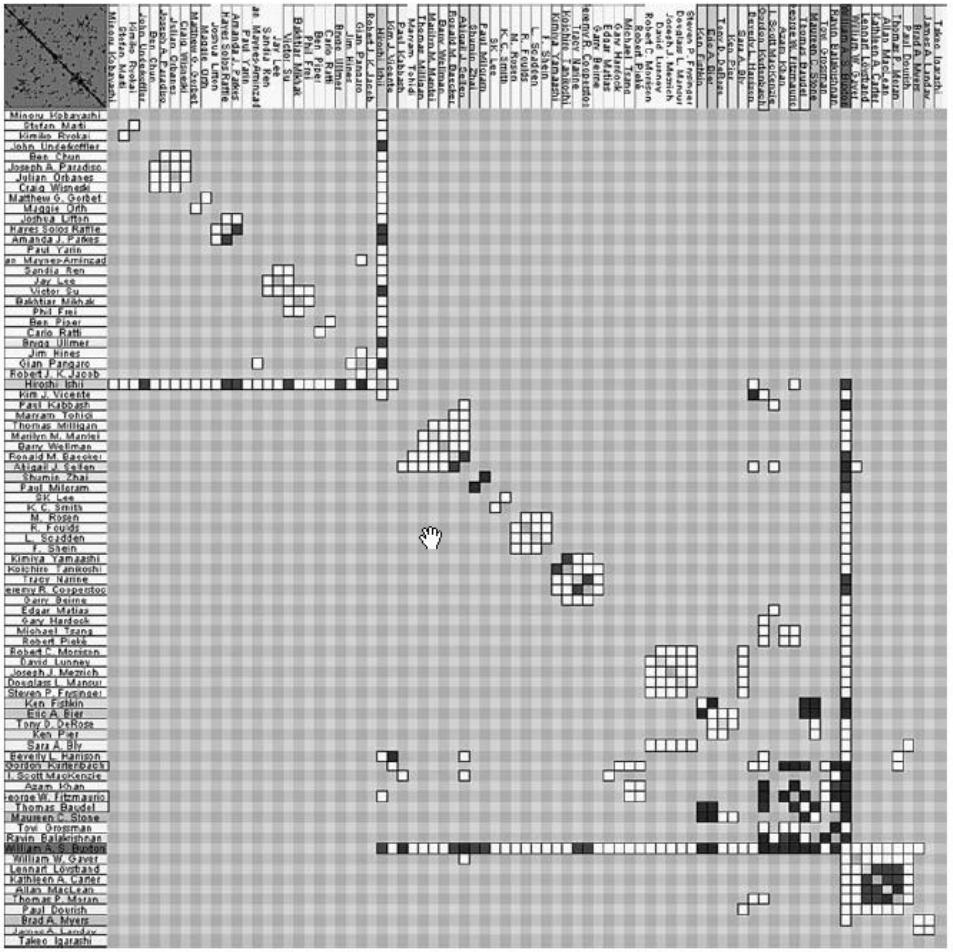
- Node-link diagrams doesn't scale well with increasing number of nodes and edges due to edge-crossings, occlusion, etc.
- There is one network representation that scales well
  - Adjacency matrix of the network
- The idea behind matrix-based visualization is to
  - Show the network as table representing the adjacency matrix
  - Nodes are rows/columns of the table
  - Edges are cells of the table



VisAdj, Ghoniem et al. 04

Matrix-based visualization of networks

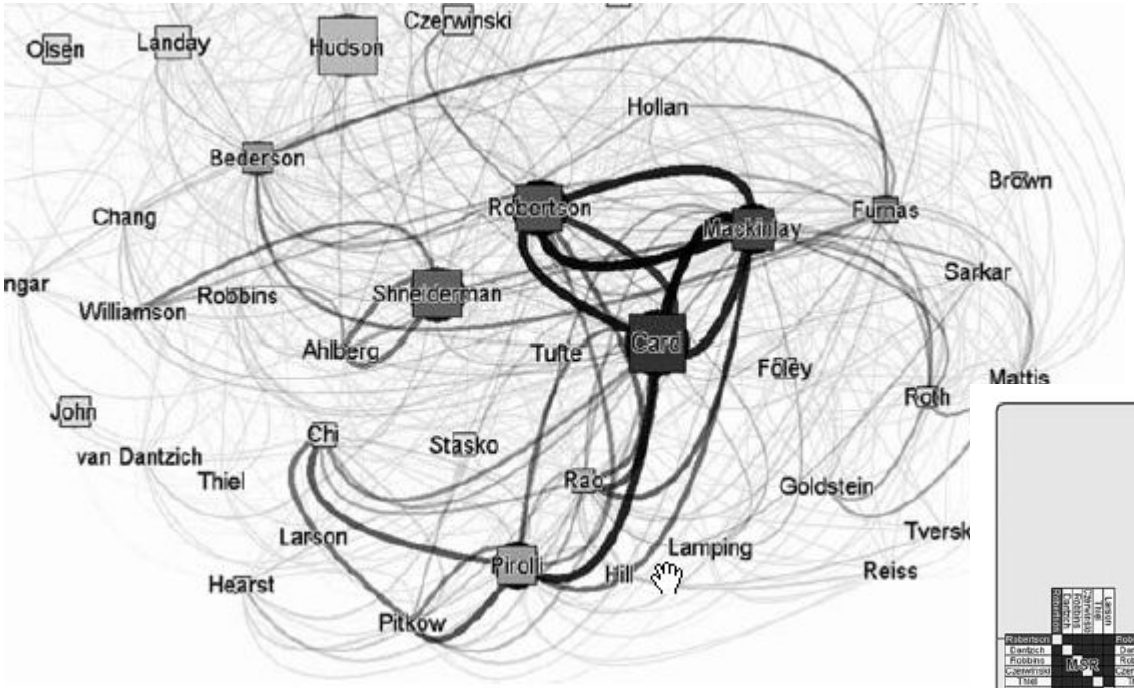
- Visualization of cooperation of authors on scientific papers from 20 years of 5 HCI conferences
- Row and column reordering is used to reveal clusters of cooperating authors

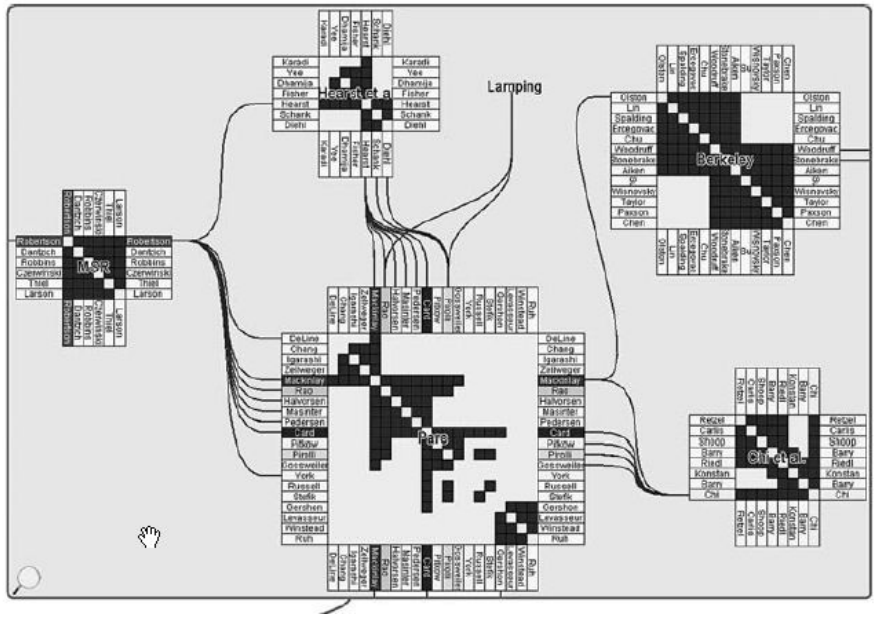


http://www.informaworld.com/smpp/content~content=a789632485~db=al

Matrices with Submatrices

http://www.informaworld.com/smpp/content~content=a789632485~db=al

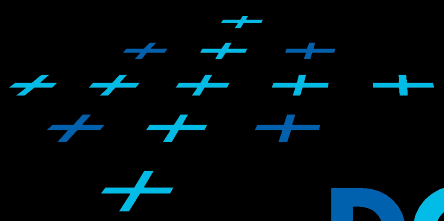




Summary

- Relational data
- What is a relation?
- Visual encoding of relations
- Visual encoding of attributes
- Examples of relational data
- Tasks for relational data
- Visualization of hierarchies with containment
- Visualization of hierarchies and networks with graphs

Thank you for your attention



DCGI